



Ch 2: Knowledge Representation and Reasoning

Artificial Intelligence

Dr. Udaya R. Dhungana
Assistant Professor
Pokhara University, Nepal

Former Guest Faculty, Darmstadt University of Applied Sciences, Germany



Overview

Ch 2: Knowledge Representation and Reasoning (7 hrs)

1. First-Order Logic (FOL) and reasoning
2. Probabilistic reasoning
 - 2.1.1. Hidden Markov Models
 - 2.1.2. Dynamic Bayesian Networks
3. Ontological Engineering
4. Semantic Networks
5. Description Logic
6. Fuzzy Logic-
 - 6.1.1. Fuzzy Inference Systems
 - 6.1.1.1. Mamdani
 - 6.1.1.2. Sugeno
 - 6.1.1.3. Tsukamoto
 - 6.1.2. Defuzzification techniques

1. Knowledge Representation (KR)

2. Logics in KR

2.1. Propositional Logic

2.2. First-Order Logic (FOL)

2.3. Description Logic (DL)

2.4. Modal Logic

2.5. Temporal Logic

2.6. Fuzzy Logic

3. Other Representation

3.1. Semantic Networks

3.2. Frames

3.3. Production Rules

4. Ontology Engineering

KNOWLEDGE REPRESENTATION AND REASONING

Knowledge Representation

- Knowledge Representation (KR)
 - is a subfield of Artificial Intelligence
 - Concerns with how knowledge can be formally structured to enable reasoning, decision-making, and learning
 - is a way to encode knowledge so that an AI system can reason about it.
- **Goal:** develop formalisms for providing *high-level descriptions* of the world that can be effectively used to build intelligent applications
- **High-level descriptions:** concentrate on representing relevant aspects for a given application, while ignoring irrelevant details.



Knowledge Representation

- To solve the problems in a domain, we need
 - **Knowledge** about the domain
 - Some mechanisms for representing that **knowledge** (facts) in computer so that the **knowledge** can be manipulated to create solutions to new problems
- **Knowledge Representation involves**
 - ***Symbols***: Used to denote objects, relations, events, etc.
 - ***Structures***: Combine symbols to represent complex knowledge (e.g., rules).
 - ***Semantics***: Define the meaning of symbols and structures.
- **Goals of KR**
 - ***Expressiveness***: Represent a wide range of knowledge
 - ***Reasoning***: Draw conclusions from the knowledge
 - ***Efficiency***: Support fast and scalable inference



Knowledge Representation

- Knowledge representation deals with
 - Facts (*knowledge level*)
 - are the truths in some relevant world
 - are need to be described first
 - these are the things we want to represent
 - Representation of facts (*symbol level*)
 - is some mechanism to encode the facts in terms of symbols
 - These are the things that can be manipulated by programs to reason and solve the problem



Properties of a Good KR System

- **Representational Adequacy**
 - The system must be able to represent all kinds of knowledge that an intelligent agent may need
 - It should support both **declarative knowledge** (facts, concepts) and **procedural knowledge** (how-to, rules, actions)
- **Inferential Adequacy**
 - the ability to derive new facts or knowledge from existing ones using automated reasoning.
 - It must support different types of reasoning: **deductive**, **inductive**, **abductive**, and **non-monotonic**



Properties of a Good KR System

- **Inferential Efficiency**
 - The system must not only support inference but do it **efficiently**
 - It should provide guidance for which inferences are most relevant and avoid combinatorial explosion
- **Acquisitional Efficiency**
 - It should be easy to acquire, encode, and modify knowledge
 - Knowledge engineers or the system itself should be able to **add new knowledge** without reworking the whole system



Properties of a Good KR System

- Expressiveness
- Compactness (Non-redundant)
- Understandable to humans (can be traced how conclusions were derived)
- Modularity
- Correctness and Consistency
- Updateability
- Support for Reasoning on Uncertainty



KR Systems

- There are multiple forms of knowledge (declarative, procedural, semantic, episodic, etc.)
- Each may require different representational strategies.
- Common KR techniques include:
 - Propositional Logic
 - First-Order Logic (Predicate Logic)
 - Semantic Networks
 - Frames
 - Ontologies
 - Rules and Production Systems
 - Description Logics



Propositional Logic

- Deals with the statements or propositions and connections between them
- Represents facts using atomic propositions (true/false statements)
 - A complete statement is represented by a single symbol
 - It does not consider smaller parts of statements and treat simple statements as indivisible (atomic)
- **Example:** represent the fact- "If it is raining, then the ground is wet"
 - P: It is raining
 - Q: The ground is wet.
 - $P \rightarrow Q$: If it is raining, then the ground is wet



Propositional Logic

- Deals with the propositions and connections between them
 - A proposition is a declarative statement that can be either true or false (**but not both**)
- Represents facts using atomic propositions (true/false statements)
 - A complete statement is represented by a single symbol
 - It does not consider smaller parts of statements and treat simple statements as indivisible (atomic)
- **Example:** represent the fact- "If it is raining, then the ground is wet"
 - P: It is raining
 - Q: The ground is wet.
 - $P \rightarrow Q$: If it is raining, then the ground is wet



Propositional Logic

Example of propositions:

John loves CSE 191.

$2+3=5$.

$2+3=8$.

Sun rises from West.

Example of non-propositions:

Does John love CSE 191?

$2 + 3$.

Solve the equation $2 + x = 3$.

$2 + x > 8$.



Propositional Logic

- **Syntax**

- The symbols in this logic are

1. **Uppercase letter** (may also contain other letters or subscripts) such as P, Q, R, $W_{1,3}$ and North

- which represent propositions such as: "It is raining" and "I am wet"

2. **connectives**

- and ($\wedge$), or ($\vee$), implies ($\rightarrow$), not ($\neg$), if and only if ($\leftrightarrow$)

3. **Parenthesis: ()**

4. **Logical constants: True and False**



Propositional Logic

Figure 7.7

$$\begin{aligned} \text{Sentence} &\rightarrow \text{AtomicSentence} \mid \text{ComplexSentence} \\ \text{AtomicSentence} &\rightarrow \text{True} \mid \text{False} \mid P \mid Q \mid R \mid \dots \\ \text{ComplexSentence} &\rightarrow (\text{Sentence}) \\ &\mid \neg \text{Sentence} \\ &\mid \text{Sentence} \wedge \text{Sentence} \\ &\mid \text{Sentence} \vee \text{Sentence} \\ &\mid \text{Sentence} \Rightarrow \text{Sentence} \\ &\mid \text{Sentence} \Leftrightarrow \text{Sentence} \end{aligned}$$

OPERATOR PRECEDENCE : $\neg, \wedge, \vee, \Rightarrow, \Leftrightarrow$

A BNF (Backus–Naur Form) grammar of sentences in propositional logic, along with operator precedences, from highest to lowest.



Propositional Logic

- **Syntax**

- Truth Tables: A truth table is a table showing the truth value of a propositional logic formula as a function of its inputs.
- Truth tables for all connectives

P	Q	$\neg P$	$P \wedge Q$	$P \vee Q$	$P \Rightarrow Q$	$P \Leftrightarrow Q$
false	false	true	false	false	true	true
false	true	true	false	true	true	false
true	false	false	false	true	false	false
true	true	false	true	true	true	true



Propositional Logic

$$\begin{aligned}(\alpha \wedge \beta) &\equiv (\beta \wedge \alpha) && \text{commutativity of } \wedge \\(\alpha \vee \beta) &\equiv (\beta \vee \alpha) && \text{commutativity of } \vee \\((\alpha \wedge \beta) \wedge \gamma) &\equiv (\alpha \wedge (\beta \wedge \gamma)) && \text{associativity of } \wedge \\((\alpha \vee \beta) \vee \gamma) &\equiv (\alpha \vee (\beta \vee \gamma)) && \text{associativity of } \vee \\\neg(\neg\alpha) &\equiv \alpha && \text{double-negation elimination} \\(\alpha \Rightarrow \beta) &\equiv (\neg\beta \Rightarrow \neg\alpha) && \text{contraposition} \\(\alpha \Rightarrow \beta) &\equiv (\neg\alpha \vee \beta) && \text{implication elimination} \\(\alpha \Leftrightarrow \beta) &\equiv ((\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)) && \text{biconditional elimination} \\\neg(\alpha \wedge \beta) &\equiv (\neg\alpha \vee \neg\beta) && \text{De Morgan} \\\neg(\alpha \vee \beta) &\equiv (\neg\alpha \wedge \neg\beta) && \text{De Morgan} \\(\alpha \wedge (\beta \vee \gamma)) &\equiv ((\alpha \wedge \beta) \vee (\alpha \wedge \gamma)) && \text{distributivity of } \wedge \text{ over } \vee \\(\alpha \vee (\beta \wedge \gamma)) &\equiv ((\alpha \vee \beta) \wedge (\alpha \vee \gamma)) && \text{distributivity of } \vee \text{ over } \wedge\end{aligned}$$

Standard logical equivalences. The symbols α , β , and γ stand for arbitrary sentences of propositional logic.



Inference in Propositional Logic

- Inference is the process of
 - deriving new sentences (conclusions) from existing sentences (premises) using logical rules
- Several inference methods are used to perform this reasoning, both
 - **syntactic** (based on structure) and
 - **semantic** (based on meaning or truth values)
- Inference Methods
 - Truth Table Approach (Semantic Approach)
 - Inference Rules Approach (Syntactic/Deductive Approach)
 - Resolution-Refutation Approach (Proof by Contradiction)



Truth-Table Approach

- A two steps procedure:
 1. Generate table for all possible **truth value assignments** for the propositional variables
 2. Check whether the **conclusion** (query α) evaluates to true whenever premises (KB) evaluates to true

Example: $KB = (A \vee C) \wedge (B \vee \neg C)$ $\alpha = (A \vee B)$

A	B	C	$A \vee C$	$(B \vee \neg C)$	KB	α
<i>True</i>	<i>True</i>	<i>True</i>				
<i>True</i>	<i>True</i>	<i>False</i>				
<i>True</i>	<i>False</i>	<i>True</i>				
<i>True</i>	<i>False</i>	<i>False</i>				
<i>False</i>	<i>True</i>	<i>True</i>				
<i>False</i>	<i>True</i>	<i>False</i>				
<i>False</i>	<i>False</i>	<i>True</i>				
<i>False</i>	<i>False</i>	<i>False</i>				



Truth-Table Approach

- A two steps procedure:
 1. Generate table for all possible **truth value assignments** for the propositional variables
 2. Check whether the **conclusion** (query α) evaluates to true whenever premises (KB) evaluates to true

Example: $KB = (A \vee C) \wedge (B \vee \neg C)$ $\alpha = (A \vee B)$

A	B	C	$A \vee C$	$(B \vee \neg C)$	KB	α
<i>True</i>	<i>True</i>	<i>True</i>				
<i>True</i>	<i>True</i>	<i>False</i>				
<i>True</i>	<i>False</i>	<i>True</i>				
<i>True</i>	<i>False</i>	<i>False</i>				
<i>False</i>	<i>True</i>	<i>True</i>				
<i>False</i>	<i>True</i>	<i>False</i>				
<i>False</i>	<i>False</i>	<i>True</i>				
<i>False</i>	<i>False</i>	<i>False</i>				

Construct a truth table for all variables A, B, C , $A \vee C, B \vee \neg C$, etc. and check if in **every row** where KB is true, the query α is also true.



Truth-Table Approach

- A two steps procedure:
 1. Generate table for all possible **truth value assignments** for the propositional variables
 2. Check whether the **conclusion** (query α) evaluates to true whenever premises (KB) evaluates to true

Example: $KB = (A \vee C) \wedge (B \vee \neg C)$ $\alpha = (A \vee B)$

A	B	C	$A \vee C$	$(B \vee \neg C)$	KB	α
<i>True</i>	<i>True</i>	<i>True</i>	<i>True</i>	<i>True</i>	<i>True</i>	<i>True</i>
<i>True</i>	<i>True</i>	<i>False</i>	<i>True</i>	<i>True</i>	<i>True</i>	<i>True</i>
<i>True</i>	<i>False</i>	<i>True</i>	<i>True</i>	<i>False</i>	<i>False</i>	<i>True</i>
<i>True</i>	<i>False</i>	<i>False</i>	<i>True</i>	<i>True</i>	<i>True</i>	<i>True</i>
<i>False</i>	<i>True</i>	<i>True</i>	<i>True</i>	<i>True</i>	<i>True</i>	<i>True</i>
<i>False</i>	<i>True</i>	<i>False</i>	<i>False</i>	<i>True</i>	<i>False</i>	<i>True</i>
<i>False</i>	<i>False</i>	<i>True</i>	<i>True</i>	<i>False</i>	<i>False</i>	<i>False</i>
<i>False</i>	<i>False</i>	<i>False</i>	<i>False</i>	<i>True</i>	<i>False</i>	<i>False</i>



Truth-Table Approach

- A two steps procedure:
 1. Generate table for all possible **truth value assignments** for the propositional variables
 2. Check whether the **conclusion** (query α) evaluates to true whenever premises (KB) evaluates to true

Example: $KB = (A \vee C) \wedge (B \vee \neg C)$ $\alpha = (A \vee B)$

<i>A</i>	<i>B</i>	<i>C</i>	$A \vee C$	$(B \vee \neg C)$	<i>KB</i>	α
<i>True</i>	<i>True</i>	<i>True</i>	<i>True</i>	<i>True</i>	<i>True</i>	<i>True</i>
<i>True</i>	<i>True</i>	<i>False</i>	<i>True</i>	<i>True</i>	<i>True</i>	<i>True</i>
<i>True</i>	<i>False</i>	<i>True</i>	<i>True</i>	<i>False</i>	<i>False</i>	<i>True</i>
<i>True</i>	<i>False</i>	<i>False</i>	<i>True</i>	<i>True</i>	<i>True</i>	<i>True</i>
<i>False</i>	<i>True</i>	<i>True</i>	<i>True</i>	<i>True</i>	<i>True</i>	<i>True</i>
<i>False</i>	<i>True</i>	<i>False</i>	<i>False</i>	<i>True</i>	<i>False</i>	<i>True</i>
<i>False</i>	<i>False</i>	<i>True</i>	<i>True</i>	<i>False</i>	<i>False</i>	<i>False</i>
<i>False</i>	<i>False</i>	<i>False</i>	<i>False</i>	<i>True</i>	<i>False</i>	<i>False</i>



Truth-Table Approach

$$KB = (A \vee C) \wedge (B \vee \neg C) \quad \alpha = (A \vee B)$$

A	B	C	$A \vee C$	$(B \vee \neg C)$	KB	α
<i>True</i>	<i>True</i>	<i>True</i>	<i>True</i>	<i>True</i>	<i>True</i>	<i>True</i>
<i>True</i>	<i>True</i>	<i>False</i>	<i>True</i>	<i>True</i>	<i>True</i>	<i>True</i>
<i>True</i>	<i>False</i>	<i>True</i>	<i>True</i>	<i>False</i>	<i>False</i>	<i>True</i>
<i>True</i>	<i>False</i>	<i>False</i>	<i>True</i>	<i>True</i>	<i>True</i>	<i>True</i>
<i>False</i>	<i>True</i>	<i>True</i>	<i>True</i>	<i>True</i>	<i>True</i>	<i>True</i>
<i>False</i>	<i>True</i>	<i>False</i>	<i>False</i>	<i>True</i>	<i>False</i>	<i>True</i>
<i>False</i>	<i>False</i>	<i>True</i>	<i>True</i>	<i>False</i>	<i>False</i>	<i>False</i>
<i>False</i>	<i>False</i>	<i>False</i>	<i>False</i>	<i>True</i>	<i>False</i>	<i>False</i>

$\therefore KB$ entails α

- **Disadvantage:**
 - Exponential time complexity: 2^n for n variables
 - Inefficient for large knowledge bases
- Advantage:
 - This approach is **sound and complete**



Truth-Table Approach

Property	Meaning	Formal Statement
Soundness	No wrong conclusions are derived	$KB \vdash \alpha \Rightarrow KB \models \alpha$
Completeness	All true conclusions can be derived	$KB \models \alpha \Rightarrow KB \vdash \alpha$



Truth-Table Approach

- **Problem:**
 - Ram was searching a job and attempted two interviews. In attempting the interview, he found the two statements written in the two different companies. In company A, it was written “If you are qualified, you will get the job.” and in company B, it was written “If you did not get the job, you were not qualified.” Does both statements are saying the same thing?
- **Solution: ??**



Truth-Table Approach

- **Solution:**
 - Company A: “If you are qualified, you will get the job.”
 - Company B: “If you did not get the job, you were not qualified.”
- **Define Propositions:** Let
 - Q: You are Qualified
 - J: You get the Job
- **Then the statement of company A and B becomes**
 - Company A: $Q \implies J$
 - Company B: $\neg J \implies \neg Q$



Truth-Table Approach

- **Solution:**

- Are the statements $Q \implies J$ and $\neg J \implies \neg Q$ logically equivalent?
- Draw the truth table:

Q	J	$\neg J$	$\neg Q$	$Q \rightarrow J$	$\neg J \rightarrow \neg Q$
F	F	T	T		
F	T	F	T		
T	F	T	F		
T	T	F	F		



Truth-Table Approach

- **Solution:**

- Are the statements $Q \implies J$ and $\neg J \implies \neg Q$ logically equivalent?
- Draw the truth table:

Q	J	$\neg J$	$\neg Q$	$Q \rightarrow J$	$\neg J \rightarrow \neg Q$
F	F	T	T	T	T
F	T	F	T	T	T
T	F	T	F	F	F
T	T	F	F	T	T



Truth-Table Approach

- **Solution:**

- Are the statements $Q \implies J$ and $\neg J \implies \neg Q$ logically equivalent?
- Draw the truth table:

Q	J	$\neg J$	$\neg Q$	$Q \rightarrow J$	$\neg J \rightarrow \neg Q$
F	F	T	T	T	T
F	T	F	T	T	T
T	F	T	F	F	F
T	T	F	F	T	T

- Since the two statements have the same truth value
 - Both statements are saying same (both are equivalent)



Inference Rules Approach

- Disadvantage of Truth-Table Approach:
 - Exponential time complexity: 2^n for n variables
 - Inefficient for large knowledge bases
- How to make the process more efficient?
 - check only entries for which KB is True
 - This is the idea behind the **inference rules approach**



Inference Rules

- Inference rules are **logical rules** used to **derive conclusions** from premises
- They are the building blocks of **deductive reasoning**, especially in **automated reasoning** and **expert systems** (like in Prolog)
- An **inference rule** takes one or more **premises** (assumed to be true) and produces a **conclusion** that must also be true if the premises are true.

- Logical Notation:

$$\bullet \frac{P_1, P_2, \dots, P_n}{C},$$

- Meaning: from premises $P_1, P_2, \dots, P_n$ infer conclusion C



Inference Rules

- **Modus Ponens**

- If both sentences in the premise are true then conclusion is true.

$$\left. \begin{array}{l} P \implies Q \\ P \end{array} \right\} \text{Premise}$$
$$\frac{P}{\therefore Q} \left\} \text{Conclusion}$$

Suppose

P : It is snowing

Q : I will study AI

Then

$P \implies Q$: “If it is snowing, then I will study AI”

P : “It is snowing”

$\therefore Q$: “I will Study AI”



Inference Rules

- **Modus Tollens**

$$\left. \begin{array}{l} P \implies Q \\ \neg Q \end{array} \right\} \text{Premise}$$
$$\frac{\neg Q}{\therefore \neg P} \left. \right\} \text{Conclusion}$$

Suppose

P : It is snowing

Q : I will study AI

Then

$P \implies Q$: “If it is snowing, then I will study AI”

$\neg Q$: “I will not study AI”

$\therefore \neg P$: “It is not snowing”



Inference Rules

- Other inference rules

Rule of inference	Form	Example
Hypothetical syllogism	$\begin{array}{l} P \rightarrow Q \\ Q \rightarrow R \\ \hline P \rightarrow R \end{array}$	If Leo is a lion, then Leo roars. If Leo roars, then Leo is fierce. Therefore, if Leo is a lion, then Leo is fierce.
Disjunctive syllogism	$\begin{array}{l} P \vee Q \\ \neg P \\ \hline Q \end{array}$	The book is on the shelf or on the table. The book is not on the shelf. Therefore, the book is on the table.
Double negation elimination	$\begin{array}{l} \neg \neg P \\ \hline P \end{array}$	We were not unable to meet the deadline. We were able to meet the deadline.



Inference Rules

Rule of inference	Form	Example
Addition $P \implies (P \vee Q)$ (OR-Introduction)	$\frac{P}{\therefore P \vee Q}$	“I will study AI.” “Therefore, I will study AI or I will visit PKR.”
Simplification $(P \wedge Q) \implies P$ (And-Elimination)	$\frac{P \wedge Q}{\therefore P}$	“I will study AI and English” “Therefore, I will study AI.”
Conjunction $((P) \wedge (Q)) \implies (P \wedge Q)$ (And-Introduction)	$\frac{P \quad Q}{P \wedge Q}$	“I will study AI.” “I will study English.” “Therefore, I will study AI and English.”
Resolution $((\neg P \vee R) \wedge (P \vee Q)) \implies (R \wedge Q)$	$\frac{\neg P \vee R \quad P \vee Q}{R \vee Q}$	“I will not study AI or I will study C” “I will study AI or I will study DBMS” “Therefore, I will study C or I will study DBMS”



Inference Rules

- **Problem:** From the proposition $P \wedge (P \rightarrow Q)$, show that Q is the conclusion
- **Solution:**

Step	Reason
1. $p \wedge (p \rightarrow q)$	Premise
2. p	Simplification using (1)
3. $p \rightarrow q$	Simplification using (1)
4. q	Modus Ponens using (2) and (3)



Inference Rules

- **Problem:** From the proposition $P \wedge (P \rightarrow Q)$, show that Q is the conclusion

- **Solution:**

A valid argument is a sequence of statements. Each statement is either a premise or follows from previous statements by rules of inference. The last statement is called conclusion.

Step	Reason
1. $p \wedge (p \rightarrow q)$	Premise
2. p	Simplification using (1)
3. $p \rightarrow q$	Simplification using (1)
4. q	Modus Ponens using (2) and (3)

Valid argument



Inference Rules

- **Problem:** With these hypotheses:
 - “It is not sunny this afternoon and it is colder than yesterday.”
 - “We will go swimming only if it is sunny.”
 - “If we do not go swimming, then we will take a canoe trip.”
 - “If we take a canoe trip, then we will be home by sunset.”
- Using the inference rules, construct a valid argument for the conclusion:
 - “We will be home by sunset.”



Inference Rules

- **Solution:**

1. Choose propositional variables:

p : "It is sunny this afternoon." r : "We will go swimming." t : "We will be home by sunset."

q : "It is colder than yesterday." s : "We will take a canoe trip."

2. Translation into propositional logic:

Hypotheses: $\neg p \wedge q, r \rightarrow p, \neg r \rightarrow s, s \rightarrow t$

Conclusion: t



Inference Rules

- **Solution:**

Step	Reason
1. $\neg p \wedge q$	Premise
2. $\neg p$	Simplification using (1)
3. $r \rightarrow p$	Premise
4. $\neg r$	Modus tollens using (2) and (3)
5. $\neg r \rightarrow s$	Premise
6. s	Modus ponens using (4) and (5)
7. $s \rightarrow t$	Premise
8. t	Modus ponens using (6) and (7)



Inference Rules

- Problem: Given $KB : P \wedge Q, P \implies R, (Q \wedge R) \implies S$. Prove Theorem: S

- **Solution:**

- | | |
|------------------------------|-------------------------------|
| 1. $P \wedge Q$ | |
| 2. $P \implies R$ | |
| 3. $(Q \wedge R) \implies S$ | |
| 4. P | From 1 and And-elim |
| 5. R | From 2,4 and Modus ponens |
| 6. Q | From 1 and And-elim |
| 7. $(Q \wedge R)$ | From 5,6 and And-introduction |
| 8. S | From 7,3 and Modus ponens |

Proved: S



Sound Inference Rule

- A **sound inference** rule is a logical rule that guarantees truth preservation:
 - If the premises are true, then the conclusion derived using the rule is also true.
- An inference rule is **sound** if it preserves truth in **all models**.
 - If $P_1, P_2, \dots, P_n \models C$, then the inference rule is sound.
 - This means whenever you apply the rule to **true premises**, the **conclusion is logically valid and cannot be false**.
- Note $KB \models \alpha$ means if the KB is true, then α must also be true under every possible interpretation (model)



Sound/ Unsound/ Complete Inference Rule

Term	Meaning
Sound	True premises $\Rightarrow$ true conclusion (no false conclusions)
Unsound	May derive false conclusions even if premises are true
Complete	Can derive all conclusions that are logically valid



Resolution-Refutation (Proof by Contradiction)

- Fundamental inference method used in
 - automated reasoning
 - theorem proving
 - logic programming
- It is especially useful in
 - Propositional Logic
 - First-Order Logic (FOL)
- The core idea is to prove the negation of a goal leads to a contradiction, thereby concluding that the goal must be true.



Resolution-Refutation (Proof by Contradiction)

- Resolution

- is a single inference rule that allows deriving a conclusion by eliminating a complementary pair of literals from two clauses.
- Example: Consider two clauses:
- Clause 1: $P \vee Q$
- Clause 2: $\neg Q \vee R$
- After Resolution: $P \vee R$



Resolution-Refutation (Proof by Contradiction)

- ***Refutation***

- is a proof method where we
- Negate the ***statement*** (goal) we want to prove
- Add it to the **knowledge base (KB)**
- Apply **resolution rules** to derive new clauses
- If we derive the empty clause **[]** (a contradiction), it proves that the original ***statement*** must be true.



Resolution-Refutation (Proof by Contradiction)

- *Steps of Resolution-Refutation:*
 - Let's say we want to prove α from KB :
 - Convert KB and $\neg \alpha$ into Conjunctive Normal Form (CNF)
 - Repeatedly apply the **resolution rule** to derive new clauses
 - If the empty clause is derived, then α is entailed by KB :

$$KB \models \alpha$$



Resolution-Refutation (Proof by Contradiction)

- **Resolution Algorithm:**
 - **Convert KB to the CNF form;**
 - **Apply iteratively the resolution rule** starting from $KB, \neg \alpha$ (in CNF form)
 - **Stop when:**
 - Contradiction (empty clause) is reached:
 - $A, \neg A \rightarrow \phi$
 - proves entailment.
 - No more new sentences can be derived
 - disproves it.



Resolution-Refutation (Proof by Contradiction)

- **Resolution Algorithm:**

Figure 7.13

```
function PL-RESOLUTION( $KB, \alpha$ ) returns true or false
  inputs:  $KB$ , the knowledge base, a sentence in propositional logic
            $\alpha$ , the query, a sentence in propositional logic

   $clauses \leftarrow$  the set of clauses in the CNF representation of  $KB \wedge \neg\alpha$ 
   $new \leftarrow \{\}$ 
  while true do
    for each pair of clauses  $C_i, C_j$  in  $clauses$  do
       $resolvents \leftarrow$  PL-RESOLVE( $C_i, C_j$ )
      if  $resolvents$  contains the empty clause then return true
       $new \leftarrow new \cup resolvents$ 
    if  $new \subseteq clauses$  then return false
   $clauses \leftarrow clauses \cup new$ 
```

A simple resolution algorithm for propositional logic. PL-RESOLVE returns the set of all possible clauses obtained by resolving its two inputs.



Resolution-Refutation (Proof by Contradiction)

- **Example 1:** Let us consider KB contains the following premises:

$$1. A \vee B$$

$$2. A \implies C$$

$$3. \neg B$$

- Prove: C

That is ($KB \models C$)



Resolution-Refutation (Proof by Contradiction)

- **Solution:**

- **Step 1:** Negate statement to be proved: $\neg C$
- **Step 2:** Add $\neg C$ to KB . Now KB contains the statements:

1. $A \vee B$
2. $A \implies C$
3. $\neg B$
4. $\neg C$

- **Step 3:** Convert the statements into CNF

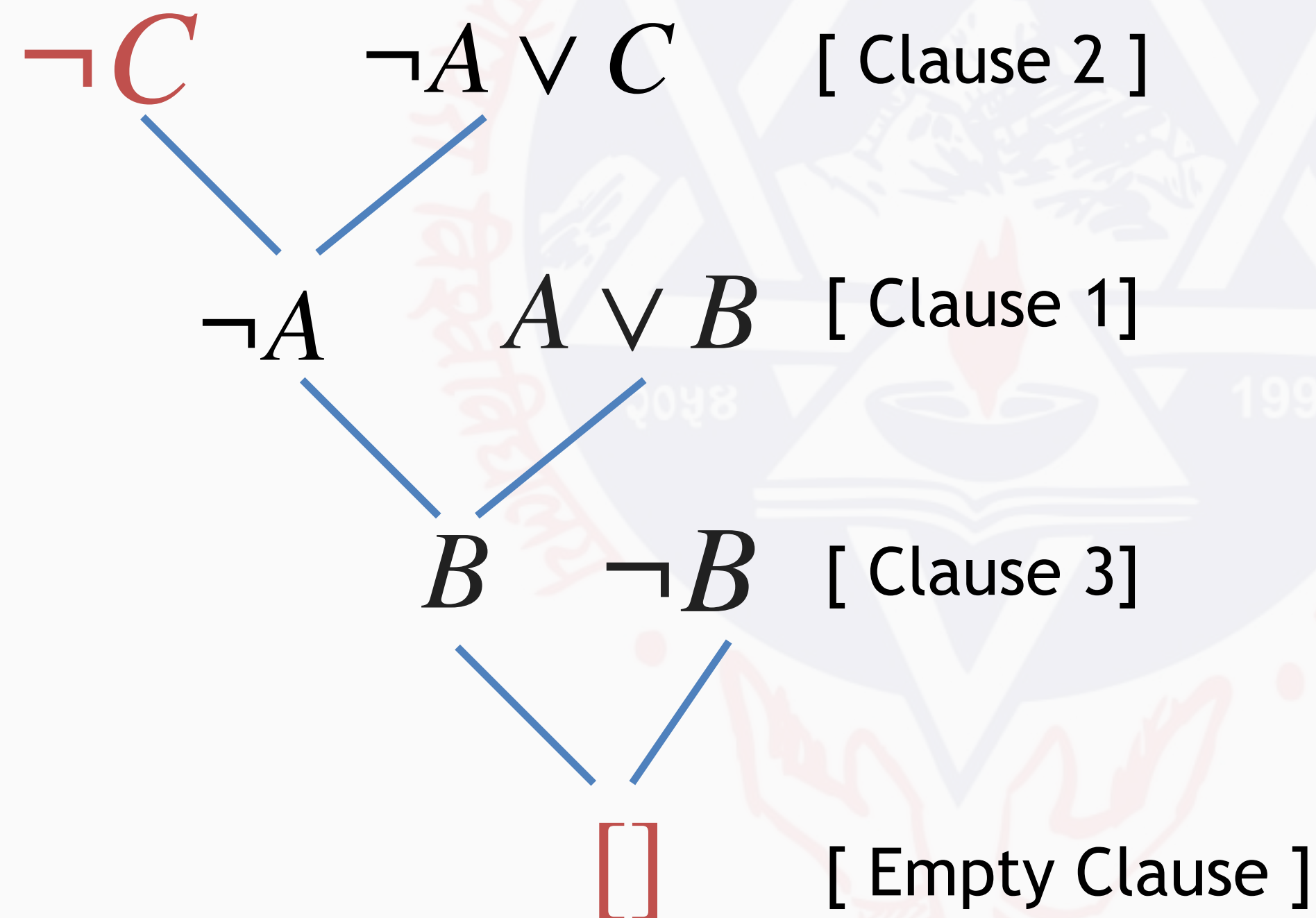
1. $A \vee B$
2. $\neg A \vee C$
3. $\neg B$
4. $\neg C$



Resolution-Refutation (Proof by Contradiction)

- **Solution:**

- **Step 4: Apply resolution**



1. $A \vee B$

2. $\neg A \vee C$

3. $\neg B$

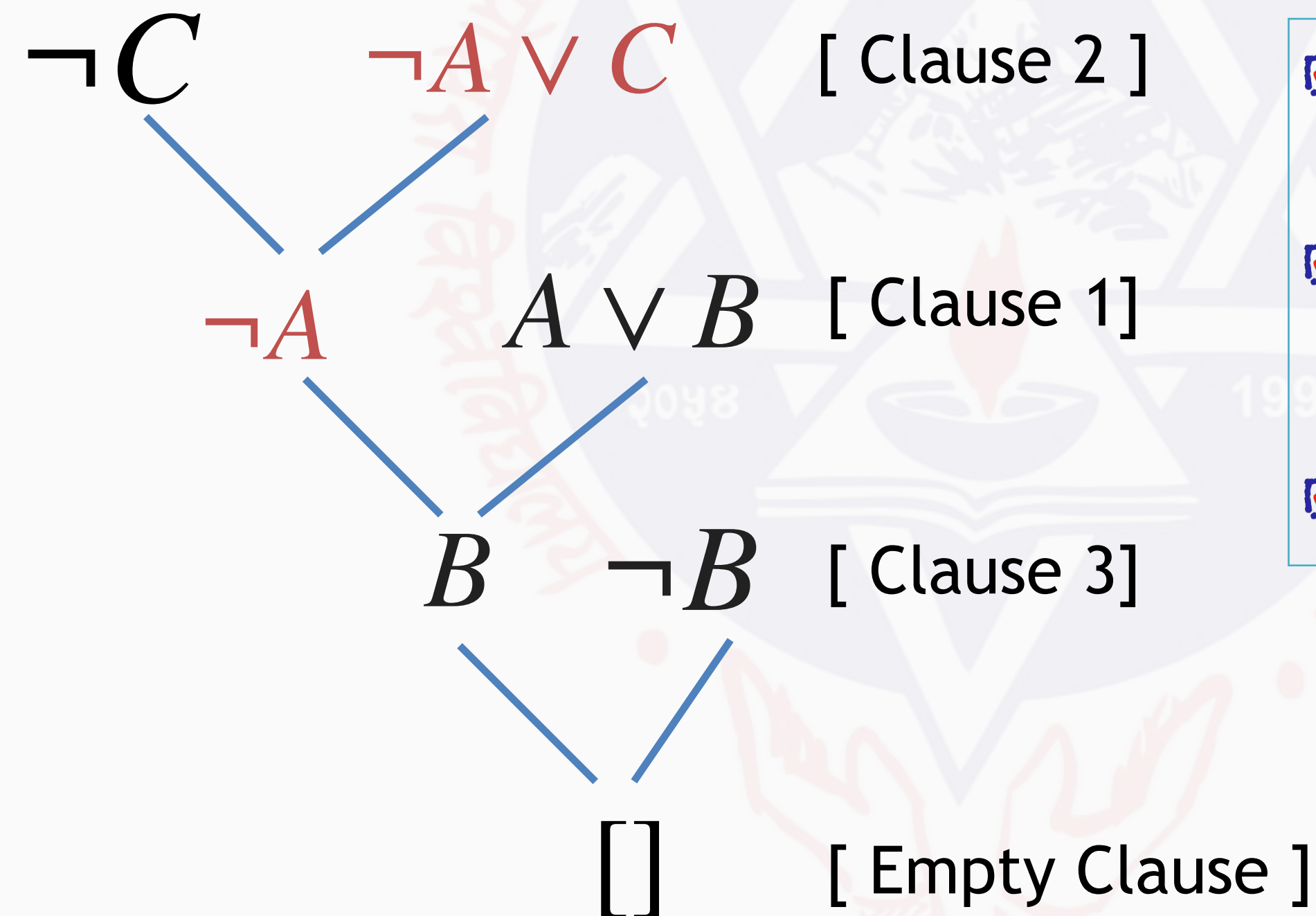
4. $\neg C$



Resolution-Refutation (Proof by Contradiction)

- **Solution:**

- **Step 4: Apply resolution**



- ✓ We started to prove the negation of C i.e $\neg C$
- ✓ During resolution, we got empty clause (Contradiction). This indicates what we are trying to prove is wrong.
- ✓ $\therefore C$ is true.

$$\therefore KB \models C$$



Resolution-Refutation (Proof by Contradiction)

- **Example 2:** KB: $(P \wedge Q) \wedge (P \Rightarrow R) \wedge [(Q \wedge R) \Rightarrow S]$ Theorem: S

Step 1. convert KB to CNF:

- $P \wedge Q \longrightarrow P \wedge Q$
- $P \Rightarrow R \longrightarrow (\neg P \vee R)$
- $(Q \wedge R) \Rightarrow S \longrightarrow (\neg Q \vee \neg R \vee S)$

KB: $P \quad Q \quad (\neg P \vee R) \quad (\neg Q \vee \neg R \vee S)$

Step 2. Negate the theorem to prove it via refutation

$S \longrightarrow \neg S$

Step 3. Run resolution on the set of clauses

$P \quad Q \quad (\neg P \vee R) \quad (\neg Q \vee \neg R \vee S) \quad \neg S$



Resolution-Refutation (Proof by Contradiction)

- *Example 2: KB: $P, Q, \neg P \vee R, \neg Q \vee \neg R \vee S, \neg S$ Theorem: S*

Class Work: Prove S .



KB in Horn Form

- **Horn form:** a clause with at most one positive literal

$$(A \vee \neg B) \wedge (\neg A \vee \neg C \vee D)$$

- **Not all sentences in propositional logic can be converted into the Horn form**
- **KB in Horn normal form:**
 - Two types of propositional statements:
 - Implications: called **rules** $(B \Rightarrow A)$
 - Propositional symbols: **facts** B
- **Application of the modus ponens:**
 - Infers new facts from previous facts

$$\frac{A \Rightarrow B, \quad A}{B}$$



Example: Forward Chaining

Assume the KB with the following rules and facts:

KB: R1: $A \wedge B \Rightarrow C$

R2: $C \wedge D \Rightarrow E$

R3: $C \wedge F \Rightarrow G$

F1: A

F2: B

F3: D

Theorem: E ?



Example: Forward Chaining

Theorem: E

KB: R1: $A \wedge B \Rightarrow C$

R2: $C \wedge D \Rightarrow E$

R3: $C \wedge F \Rightarrow G$

F1: A

F2: B

F3: D

Rule R1 is satisfied.

F4: C

KB: R1: $A \wedge B \Rightarrow C$

R2: $C \wedge D \Rightarrow E$

R3: $C \wedge F \Rightarrow G$

F1: A

F2: B

F3: D

Rule R1 is satisfied.

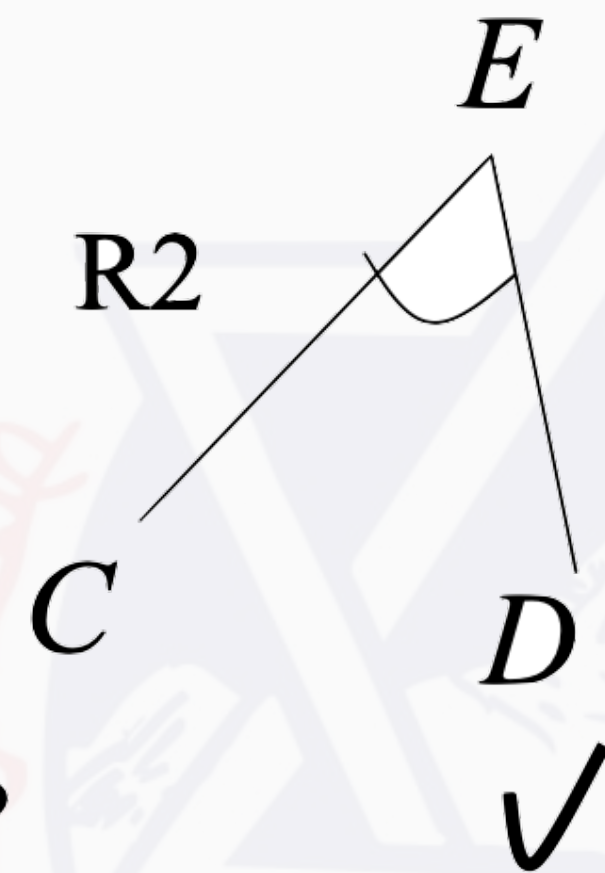
F4: C

Rule R2 is satisfied.

F5: E



Example: Backward Chaining



KB: R1: $A \wedge B \Rightarrow C$

R2: $C \wedge D \Rightarrow E$

R3: $C \wedge F \Rightarrow G$

F1: A

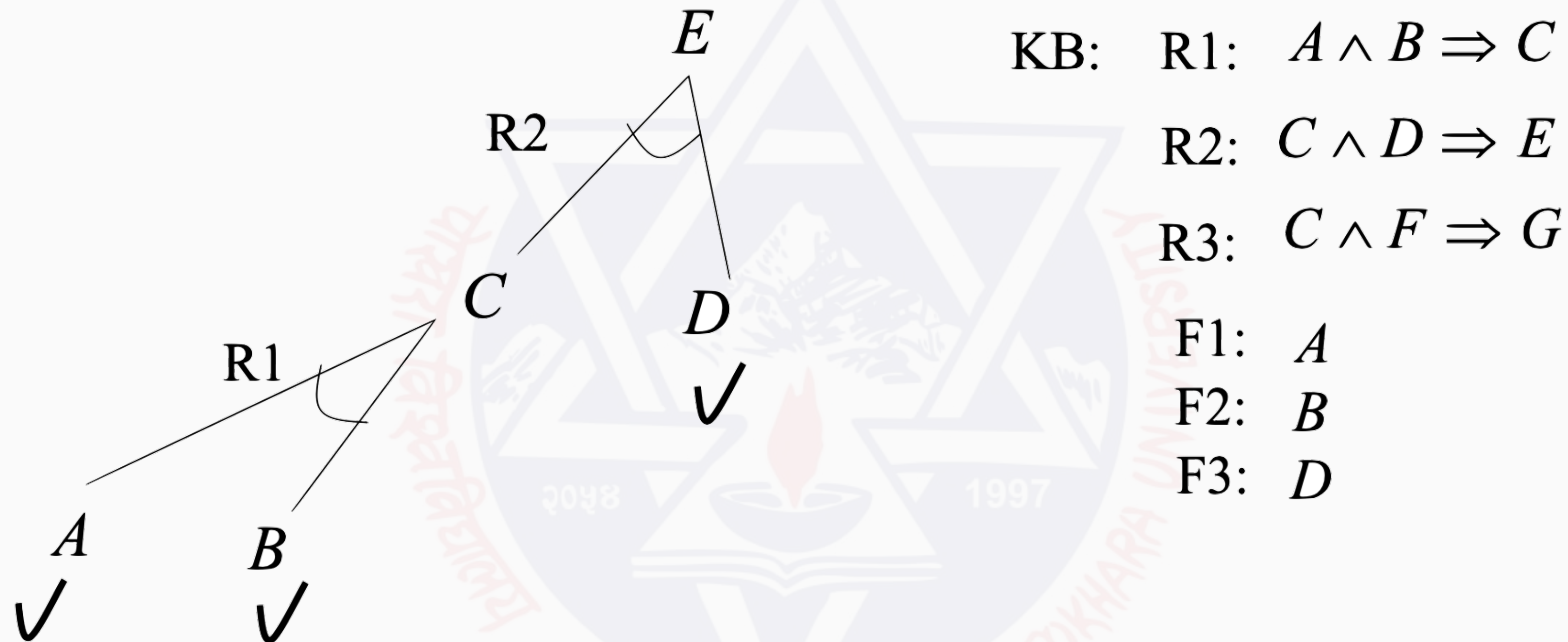
F2: B

F3: D

- Backward chaining is more focused:
 - tries to prove the theorem only



Example: Backward Chaining



- Backward chaining is more focused:
 - tries to prove the theorem only



A Knowledge-based Agent

- A Knowledge-Based Agent typically follows this cycle:
 - Perceive the environment
 - Update the Knowledge Base using the new percept
 - Infer what action to take using logical reasoning
 - Act based on the chosen action
- This is called the Perceive → Think → Act cycle.



Components of A Knowledge-based Agent

- **Knowledge Base (KB)**

- A collection of sentences expressed in a formal knowledge representation language, PL or FOL.
- These sentences represent what the agent knows.

- **Inference Engine**

- Derives conclusions from the KB using inference rules.
- Uses methods like **modus ponens**, **unification**, or **resolution**

- **Knowledge Acquisition**

- The process of getting new information from percepts and updating the KB.



A Knowledge-based Agent

Figure 7.1

```
function KB-AGENT(percept) returns an action  
  persistent: KB, a knowledge base  
             t, a counter, initially 0, indicating time  
  
  TELL(KB, MAKE-PERCEPT-SENTENCE(percept, t))  
  action ← ASK(KB, MAKE-ACTION-QUERY(t))  
  TELL(KB, MAKE-ACTION-SENTENCE(action, t))  
  t ← t + 1  
  return action
```

A generic knowledge-based agent. Given a percept, the agent adds the percept to its knowledge base, asks the knowledge base for the best action, and tells the knowledge base that it has in fact taken that action.



A Knowledge-based Agent

Figure 7.1

function KB-AGENT(*percept*) **returns** an *action*

persistent: *KB*, a knowledge base

t, a counter, initially 0, indicating time

TELL(*KB*, MAKE-PERCEPT-SENTENCE(*percept*, *t*))

action ← ASK(*KB*, MAKE-ACTION-QUERY(*t*))

TELL(*KB*, MAKE-ACTION-SENTENCE(*action*, *t*))

t ← *t* + 1

return *action*

- MAKE-PERCEPT-SENTENCE(*percept*, *t*) converts the raw percept into a logical sentence.
- TELL: Adds the sentence to the KB.
- This updates the agent's internal belief state about the world.

A generic knowledge-based agent. Given a percept, the agent adds the percept to its knowledge base, asks the knowledge base for the best action, and tells the knowledge base that it has in fact taken that action.



A Knowledge-based Agent

Figure 7.1

function KB-AGENT(*percept*) **returns** an *action*
persistent: *KB*, a knowledge base
 t, a counter, initially 0, indicating time

```
TELL(KB, MAKE-PERCEPT-SENTENCE(percept, t))  
action ← ASK(KB, MAKE-ACTION-QUERY(t))  
TELL(KB, MAKE-ACTION-SENTENCE(action, t))  
t ← t + 1  
return action
```

- MAKE-ACTION-QUERY(*t*) constructs a logical query that expresses "What should I do at time *t*?"
- TELL: Adds the sentence to the KB.
- ASK: Queries the KB and uses logical inference to return the best action based on current knowledge.
- This is the reasoning or decision-making step.

A generic knowledge-based agent. Given a percept, the agent adds the percept to its knowledge base, asks the knowledge base for the best action, and tells the knowledge base that it has in fact taken that action.



A Knowledge-based Agent

Figure 7.1

function KB-AGENT(*percept*) **returns** an *action*

persistent: *KB*, a knowledge base

t, a counter, initially 0, indicating time

TELL(*KB*, MAKE-PERCEPT-SENTENCE(*percept*, *t*))

action ← ASK(*KB*, MAKE-ACTION-QUERY(*t*))

TELL(*KB*, MAKE-ACTION-SENTENCE(*action*, *t*))

t ← *t* + 1

return *action*

- MAKE-ACTION-SENTENCE(action, t) creates a logical sentence such as: "At time t, I performed action A."
- Again, TELL adds this to the KB.
- This records the agent's own behavior into its belief system for future reasoning.

A generic knowledge-based agent. Given a percept, the agent adds the percept to its knowledge base, asks the knowledge base for the best action, and tells the knowledge base that it has in fact taken that action.



A Knowledge-based Agent

Figure 7.1

function KB-AGENT(*percept*) **returns** an *action*

persistent: *KB*, a knowledge base

t, a counter, initially 0, indicating time

TELL(*KB*, MAKE-PERCEPT-SENTENCE(*percept*, *t*))

action ← ASK(*KB*, MAKE-ACTION-QUERY(*t*))

TELL(*KB*, MAKE-ACTION-SENTENCE(*action*, *t*))

t ← *t* + 1

return *action*

- The time step increments.
- This is important because the state of the world may change over time, and so must the KB

A generic knowledge-based agent. Given a percept, the agent adds the percept to its knowledge base, asks the knowledge base for the best action, and tells the knowledge base that it has in fact taken that action.



Drawbacks of Propositional Logic

- **Lack of Expressiveness**

- **Propositional Logic** can only express facts as true or false
- It cannot represent
 - objects
 - properties
 - relations between objects
- Example:
 - In PL: *rainy* — cannot say "it is raining *in Pokhara*."
 - In FOL: *Rain(Kathmandu)*— expresses rain in a specific location.



Drawbacks of Propositional Logic

- **No Quantifiers**

- PL has no way to express **universal** or **existential** quantification like:
 - “All the students, who study, pass their exams.”
 - "John loves someone"
- In FOL, these statements are represented as
 - $\forall x: (\text{Student}(x) \wedge \text{Studies}(x) \rightarrow \text{Passes}(x))$
 - $\exists y: \text{Loves}(\text{John}, y)$
- Propositional logic has **no mechanism** to express such generalizations.



Drawbacks of Propositional Logic

- **No Variables**

- PL lacks **variables** to denote arbitrary individuals

- In PL, variables are as follows:

- likes_john_icecream
 - likes_mary_icecream
- } (separate atoms)

- In FOL, variables are as follows:

- Likes(x, IceCream)

(general rule over variables).

- You need **separate propositions** for each individual, which is impractical



Drawbacks of Propositional Logic

- **Poor Scalability**

- Every new fact requires a new propositional symbol.
- You can't reuse patterns or rules as in FOL
- PL becomes inefficient and redundant as domain size increases

- **Cannot Represent Structure of the World**

- PL treats all symbols as **atomic**, with no internal structure
- Cannot express nested or hierarchical relationships (e.g., Father(John, Mary)).

- **Difficult to Represent Knowledge Compactly**

- PL requires **enumerating** each fact explicitly
- FOL allows you to represent **many facts compactly** using variables and quantifiers



Drawbacks of PL: Examples

- **Consider the statements:**

“All men are mortal.

John is a man.

Then John is mortal”

- **Fails to capture relationship** between John and man or John and mortal.
- In addition, we do not get any information about the objects involved.
 - For example, if asked a question : **“who is a man?” we cannot get answer.**

- **Consider the statements:**

“Not all integers are even.”

“Some integers are not even”

- The Propositional logic treats statements independently.
- There is no mechanism in propositional logic to find out whether these two statements are equivalent.



First-Order Logic (FOL)

- More expressive than propositional logic
- Eliminates deficiencies of PL by:
 - Representing objects, their properties, relations and statements about them
 - Introducing variables that refer to an arbitrary objects and can be substituted by a specific object
 - Introducing quantifiers allowing us to make statements over groups objects without the need to represent each of them separately
- FOL is powerful enough for expression and reasoning.



First-Order Logic (FOL)

- PL is foundation for FOL
 - adopt the foundation of PL—a declarative, compositional semantics that is context-independent and unambiguous
 - build a more expressive logic on that foundation, borrowing representational ideas from natural language while avoiding its drawbacks.
- The syntax of natural language contains the elements:
 - Objects
 - Relations and Properties
 - Functions



First-Order Logic (FOL)

- **Objects:**

- Objects are terms (e.g. people, houses, car, John ...)

- **Relations:**

- takes terms as arguments, and results in a sentence, denoting a claim. These can be
 - Unary relations (*properties*) on terms (e.g. red, round, Hight, area,)
 - N-ary relations on terms (e.g. *father of*, *brother of*, *bigger than*, *part of* ...)

- **Functions:**

- Mapping from terms to other terms. A *function* takes terms as arguments, and returns result in another *term*, denoting an *object* (e.g. *getFather("Sasha") --> "Barack Obama"* , *plus (1, 2)* ...)
- The language of **FOL**, whose syntax and semantics is built around objects and relations.



First-Order Logic (FOL): Syntax

- **Constants**

- represent specific objects
- e.g. *John, Nepal, Car29, ...*

- **Variables**

- represent objects of a certain type
- e.g. *x, y, a, b, ...*

- **Relations and Functions**

- *Father(Ram, Hari)* // Binary relation
- *IsBlue(Car29)* // Unary relation (Property)
- *FatherOf(Hari)* // Function
- *Sqrt(4)* // Function

Has_color(Car29, Blue)???



First-Order Logic (FOL): Syntax

- **Logical Connectives**

- $\neg, \wedge, \vee, \rightarrow, \leftrightarrow$

- **Equality (=)**

- $t1 = t2$ // equivalence of terms
- e.g. $Ram = FatherOf(Hari)$

- **Quantifiers**

- $\forall$ (universal quantifier)
 - $\forall x : Human(x) \implies Mortal(x)$ [All Human are mortal.]
- $\exists$ (existential quantifier)
 - $\exists y : Girl(y) \implies ShortHair(y)$ [Some girl have short hair.]



First-Order Logic (FOL): Syntax

- **Atomic Sentences**

- $Father(Ram, Hari)$
- $IsBlue(Car29)$
- $Ram = FatherOf(Hari)$

- **Complex Sentences**

- Suppose α , β and γ are atomic sentences in FOL, then complex sentences are:
 - $(\alpha \vee \beta) \wedge \neg(\beta \vee \neg\gamma)$
 - $\beta \implies \gamma$
 - $\neg\alpha$
 - $\alpha \iff \beta$
 - $\forall x : Human(x) \implies Mortal(x)$
 - $\exists y : Girl(y) \implies ShortHair(y)$



First-Order Logic (FOL): Syntax

- **More on Universal Quantifier $\forall$**

- $\forall x$: means “for all” x
 - represent phrase “ for all”.
 - It says that something is true for all possible values of a variable x .
 - It corresponds to conjunction ("and") in that $\forall x : P(x)$ means that P holds for all values of x in the domain associated with that variable
 - They are usually used with "implies"

- Example

“John loves everyone”

$\forall x : \text{loves}(\text{John}, x)$

“All students who are studying at PU are smart.”

$\forall x : \text{student}(x) \wedge \text{studyIn}(x, \text{PU}) \implies \text{smart}(x)$



First-Order Logic (FOL): Syntax

- **More on Existential Quantifier $\exists$**

- $\exists x$: means “there exists some x ”
 - represent phrase “there exists some”.
 - It says that something is true for some possible values of a variable x .
 - It corresponds to disjunction ("or") in that $\exists x : P(x)$ means that P holds for some value of x in the domain associated with that variable
 - They are usually used with "and" to specify a list of properties or facts about an individual
- Example:

John loves someone

$\exists y : \text{loves}(\text{John}, y)$

Some people like reading and hence they gain knowledge.

$\exists x : \text{person}(x) \wedge \text{like}(x, \text{reading}) \wedge \text{gain}(x, \text{knowledge})$

Typically the existential quantifier connects with a conjunction.



First-Order Logic (FOL): Syntax

- Assume two predicates $S(x)$ and $P(x)$

Universal statements typically tie with implications

- All $S(x)$ is $P(x)$
 - $\forall x (S(x) \rightarrow P(x))$
- No $S(x)$ is $P(x)$
 - $\forall x (S(x) \rightarrow \neg P(x))$

Existential statements typically tie with conjunction

- Some $S(x)$ is $P(x)$
 - $\exists x (S(x) \wedge P(x))$
- Some $S(x)$ is not $P(x)$
 - $\exists x (S(x) \wedge \neg P(x))$



First-Order Logic (FOL): Syntax

- **Nesting of Quantifiers**

"Everybody loves somebody."

$\forall x : \exists y : \text{loves}(x, y)$

- **Properties of quantifiers**

$\forall x \forall y$ is the same as $\forall y \forall x$

$\exists x \exists y$ is the same as $\exists y \exists x$

$\exists x \forall y$ is not the same as $\forall y \exists x$

- Order of quantifiers of the same type does not matter
- Order of different quantifiers changes the meaning

$\forall x \exists y \text{ loves } (x, y)$
Everybody loves somebody

$\exists y \forall x \text{ loves } (x, y)$
There is someone who is loved by everyone



First-Order Logic (FOL): Syntax

- **Properties of quantifiers**

- Quantifier duality: each can be expressed using the other

$$\forall x : Likes(x, IceCream) = \neg \exists x : \neg Likes(x, IceCream)$$

$$\exists x : Likes(x, Broccoli) = \neg \forall x : \neg Likes(x, Broccoli)$$

- In General

$$\forall x : \neg P = \neg \exists x : P$$

$$\neg \forall x : P = \exists x : \neg P$$

$$\forall x : P = \neg \exists x : \neg P$$

$$\exists x : P = \neg \forall x : \neg P$$

$$\forall x : P(X) \wedge Q(X) = \forall x : P(X) \wedge \forall x : Q(X)$$

$$\exists x : P(X) \wedge Q(X) = \exists x : P(X) \wedge \exists x : Q(X)$$



Inference in First-Order Logic (FOL)

- Suppose you are given
 - a knowledge base KB (a set of sentences) and
 - a sentence α ,
 - **Inference**: does the KB semantically entail α ?

$KB \models \alpha$??



Inference in First-Order Logic (FOL)

- Is the Truth-table approach a viable approach for the FOL?

No

- Why?
 - It would require us to enumerate and list all possible interpretations I
 - I = (assignments of symbols to objects, predicates to relations and functions to relational mappings)
 - Simply there are too many interpretations



Inference in First-Order Logic (FOL)

- Is the Inference rule approach a viable approach for the FOL?

Yes

- Why?
 - The inference rules represent sound inference patterns one can apply to sentences in the KB
 - What is derived follows from the KB
 - **Note:** but we need to add rules for handling quantifiers
 - Inference Rules from PL +
 - **Additional inference rules** are needed for sentences with quantifiers and variables
 - Must involve variable substitutions



Inference Rules for Quantifiers

- Suppose our KB contains axiom: “All greedy kings are evils.”

$$\bullet \alpha = \forall x : King(x) \wedge Greedy(x) \implies Evil(x)$$

- **Universal Instantiation**

- we can infer any sentence obtained by substituting a **ground term** (a term without variables) for a universally quantified variable

Formal notation: $\frac{\forall x : \alpha}{SUBST(\{x/g\}, \alpha)}$, substitute x by ground term g . Some Examples:

$$1. \frac{\forall x : \alpha}{SUBST(\{x/Richard\}, \alpha)} = King(Richard) \wedge Greedy(Richard) \implies Evil(Richard)$$

$$2. \frac{\forall x : \alpha}{SUBST(\{x/Father(John)\}, \alpha)} = King(Father(John)) \wedge Greedy(Father(John)) \implies Evil(Father(John))$$



Inference Rules for Quantifiers

- Suppose our KB contains axiom: “All greedy kings are evils.”

- $\alpha = \exists x : \text{Crown}(x) \wedge \text{Greedy}(x)$

- **Existential Instantiation**

- replaces an existentially quantified variable with a single *new constant symbol* that does not appear elsewhere in the knowledge base

- Formal notation: $\frac{\exists x : \alpha}{\text{SUBST}(\{x/g\}, \alpha)}$, substitute x by ground term g . Some Examples:

1. $\frac{\forall x : \alpha}{\text{SUBST}(\{x/C1\}, \alpha)} = \text{Crown}(C1) \wedge \text{Greedy}(C1)$

// In logic, the new name $C1$ is called a **Skolem constant**



Unification

- **Unification** is the process of finding a **substitution** θ (called the *unifier*) that makes two expressions **syntactically identical**.
 - If such a substitution exists, the expressions are **unifiable**.
- The *UNIFY* algorithm takes two sentences and returns a **unifier** for them (a substitution) if one exists:
- $UNIFY(p, q) = \theta$ where $SUBST(\theta, p) = SUBST(\theta, q)$:

$UNIFY (Knows(John, x), Knows(John, Jane)) = \{x / Jane\}$

$UNIFY (Knows(John, x), Knows(y, Bill)) = \{x / Bill, y / John\}$

$UNIFY (Knows(John, x), Knows(y, Mother(y))) = \{y / John, x / Mother(John)\}$

$UNIFY (Knows(John, x), Knows(x, Elizabeth)) = failure.$



Unification

- **Unification** is the process of finding a **substitution** θ (called the *unifier*) that makes two expressions **syntactically identical**.
 - If such a substitution exists, the expressions are **unifiable**.

- The *UNIFY* algorithm takes two sentences and returns a **unifier** for them (a substitution) if one exists:

- $UNIFY(p, q) = \theta$ where $SUBST(\theta, p) = SUBST(\theta, q)$:

$UNIFY (Knows(John, x), Knows(John, Jane)) = \{x/Jane\}$

$UNIFY (Knows(John, x), Knows(y, Bill)) = \{x/Bill, y/John\}$

$UNIFY (Knows(John, x), Knows(y, Mother(y))) = \{y/John, x/Mother(John)\}$

$UNIFY (Knows(John, x), Knows(x, Elizabeth)) = failure.$

$\theta = \{x/Jane\}$

$\theta = \{x/Bill, y/John\}$



Unification

- For $p = \text{Knows}(\text{John}, x)$ $q = \text{Knows}(x, \text{Elizabeth})$, UNIFY algorithm fails.
 - because x can't take values John and Elizabeth at the same time
 - Problem is due to use of same variable x in both sentences
- **Simple solution:** by standardizing apart one of the two sentences being unified,
 - which means renaming its variables to avoid name clashes.
 - For example, we can rename x in $\text{Knows}(x, \text{Elizabeth})$ to x_{17} (a new variable name) without changing its meaning. Now the unification will work:
$$\text{UNIFY}(\text{Knows}(\text{John}, x), \text{Knows}(x_{17}, \text{Elizabeth})) = \{x / \text{Elizabeth}, x_{17} / \text{John}\}.$$



Inference in FOL

- **Unification** plays a central role in inference in FOL, especially in algorithms like
 - resolution
 - forward chaining
 - backward chaining



Resolution in FOL

- Produces proof by refutation (Proof by contradiction)
- To prove a statement,
 - **resolution** attempts to show the negation of the statement produces a contradiction with the known statements.
- Requires sentences to be in Conjunctive Normal Form (CNF)



Resolution in FOL

- Produces proof by refutation (Proof by contradiction)
- To prove a statement,
 - **resolution** attempts to show the negation of the statement produces a contradiction with the known statements.
- Requires sentences to be in Conjunctive Normal Form (CNF)



Resolution in FOL

- **Resolution**

AND = Conjunctions

OR = Disjunction

- **Conjunctive Normal Form (CNF)**

- In CNF, statements are conjunctions (sequence of ANDs) of clauses with clauses of disjunctions (sequence of OR).
- In other words, **a statement is a series of ORs connected by ANDs.**
- Examples:

$$(P \vee Q) \wedge (\neg P \vee R)$$



Resolution in FOL

– Conversion to CNF Form:

1. Eliminate biconditionals ($\leftrightarrow$) and implications ($\rightarrow$) using

$$p \leftrightarrow q \equiv (p \rightarrow q) \wedge (q \rightarrow p)$$

$$p \rightarrow q \equiv \neg p \vee q$$

2. Move $\neg$ inwards:

$$\neg \forall x : p(x) \equiv \exists x : \neg p(x)$$

$$\neg \exists x : p(x) \equiv \forall x : \neg p(x)$$

$$\neg(a \vee \beta) \equiv \neg a \wedge \neg \beta$$

$$\neg(a \wedge \beta) \equiv \neg a \vee \neg \beta$$

$$\neg \neg a \equiv a$$



Resolution in FOL

– Conversion to CNF Form:

3. Standardize variables so that each quantifier binds a unique variable. For instance

$\forall x : P(x) \vee \forall x : Q(x)$ could be converted into

$\forall x : P(x) \vee \forall y : Q(y)$

4. Move all quantifiers to the left without changing their relative order.

$\forall x : P(x) \vee \forall y : Q(y) \equiv$

$\forall x : \forall y : (P(x) \vee Q(y))$



Resolution in FOL

- Conversion to CNF Form:

5. Skolemize: each existential variable is replaced by a Skolem constant or Skolem function of the enclosing universally quantified variables.

1. $\exists x \text{ Rich}(x) \equiv \text{Rich}(G1)$ where G1 is a new Skolem constant.

2. $\forall x [\exists y \text{ animal}(y) \wedge \neg \text{loves}(x, y)] \vee [\exists z \text{ loves}(z, x)] \equiv$
 $\forall x [\text{animal}(F(x)) \wedge \neg \text{loves}(x, F(x))] \vee [\text{loves}(G(x), x)]$

where F and G are Skolem functions.

6. Drop universal quantifiers

$[\text{animal}(F(x)) \wedge \neg \text{loves}(x, F(x))] \vee [\text{loves}(G(x), x)]$



Resolution in FOL

– Conversion to CNF Form:

7. Distribute $\wedge$ over $\vee$

$$[\text{animal}(F(x)) \wedge \neg \text{loves}(x, F(x))] \vee [\text{loves}(G(x), x)] \equiv$$

$$[\text{animal}(F(x)) \vee \text{loves}(G(x), x)] \wedge [\neg \text{loves}(x, F(x)) \vee \text{loves}(G(x), x)]$$

- This sentence is now in CNF
- Consists of two clauses

$$[\text{animal}(F(x)) \vee \text{loves}(G(x), x)]$$

$$[\neg \text{loves}(x, F(x)) \vee \text{loves}(G(x), x)]$$

- Is quite unreadable (human seldom need look at CNF sentences)



Resolution in FOL

– Example: Conversion to CNF

“Everyone who loves all animal is loved by someone”

In predicate logic:

$$\forall x [\forall y \text{ animal } (y) \rightarrow \text{loves}(x, y)] \rightarrow [\exists y \text{ loves}(y, x)]$$

1. Eliminate implications

$$\forall x [\forall y \neg \text{animal } (y) \vee \text{loves}(x, y)] \rightarrow [\exists y \text{ loves}(y, x)] \equiv$$

$$\forall x [\neg [\forall y \neg \text{animal } (y) \vee \text{loves}(x, y)] \vee [\exists y \text{ loves}(y, x)]]$$

2. Move $\neg$ inwards

$$\forall x [\neg \forall y \neg (\neg \text{animal } (y) \vee \text{loves}(x, y))] \vee [\exists y \text{ loves}(y, x)] \equiv$$

$$\forall x [\exists y \text{ animal } (y) \wedge \neg \text{loves}(x, y)] \vee [\exists y \text{ loves}(y, x)]$$

3. Standardize variables

$$\forall x [\exists y \text{ animal } (y) \wedge \neg \text{loves}(x, y)] \vee [\exists z \text{ loves}(z, x)]$$



Resolution in FOL

4. Skolemize (process of removing existential quantifiers)

$$\forall x [\text{animal}(F(x)) \wedge \neg \text{loves}(x, F(x))] \vee \text{loves}(G(x), x)$$

Where F and G are skolem functions.

5. Drop universal quantifiers

$$[\text{animal}(F(x)) \wedge \neg \text{loves}(x, F(x))] \vee \text{loves}(G(x), x)$$

3. Distribute $\wedge$ over $\vee$

$$[\text{animal}(F(x)) \vee \text{loves}(G(x), x)] \wedge [\neg \text{loves}(x, F(x)) \vee \text{loves}(G(x), x)]$$

- This is in CNF and has two clauses

$$[\text{animal}(F(x)) \vee \text{loves}(G(x), x)]$$

$$[\neg \text{loves}(x, F(x)) \vee \text{loves}(G(x), x)]$$



Resolution in FOL

- Resolution
 - Is rule of inference
 - Pre-processing steps:
 1. Convert the given English sentence into predicate sentence.
 2. Convert all of these sentences into CNF.



Resolution in FOL

- **Resolution**

- Assume that a set of given statements F and a statement to be proved P :

- **ALGORITHM: RESOLUTION IN PREDICATE LOGIC**

1. Convert all the statements of F to clause form

2. Negate P and convert the result to clause form. Add it to the set of clauses obtained in step 1.

3. Repeat until either a contradiction is found or no progress can be made:

- a. Select two clauses. Call these the parent clauses.

- b. Resolve them together. The resulting clause is called the resolvent.

- c. If the resolvent is the empty clause, then a contradiction has been found. (It means the assumption is wrong and the original clause is true.)

If it is not then add it to the set of clauses available to the procedure.



Resolution in FOL

- **Resolution**
 - Example: Let facts about Marcus:
 1. Marcus was a man.
 2. Marcus was a Pompeian.
 3. All Pompeians were Romans.
 4. Caesar was a ruler.
 5. All Romans were either loyal to Caesar or hated him.
 6. Every one is loyal to someone.
 7. People only try to assassinate rulers they are not loyal to.
 8. Marcus tried to assassinate Caesar.
 9. All men are people



Resolution in FOL

- **Resolution**

- Example: given facts in predicate logic:

- 1. $\text{Man}(\text{Marcus})$

- 2. $\text{Pompeian}(\text{Marcus})$

- 3. $\forall x : \text{Pompeian}(x) \rightarrow \text{Roman}(x)$

- 4. $\text{ruler}(\text{Caesar})$

- 5. $\forall x : \text{Roman}(x) \rightarrow \text{loyalto}(x, \text{Caesar}) \vee \text{hate}(x, \text{Caesar})$

- 6. $\forall x : \exists y : \text{loyalto}(x, y)$

- 7. $\forall x : \forall y : \text{person}(x) \wedge \text{ruler}(y) \wedge \text{try-assassinate}(x, y) \rightarrow \neg \text{loyalto}(x, y)$

- 8. $\text{try-assassinate}(\text{Marcus}, \text{Caesar})$

- 9. $\forall x : \text{man}(x) \rightarrow \text{person}(x)$



Resolution in FOL

– Example: given facts in predicate logic:

1. $\text{Man}(\text{Marcus})$

2. $\text{Pompeian}(\text{Marcus})$

3. $\forall x : \text{Pompeian}(x) \rightarrow \text{Roman}(x)$

4. $\text{ruler}(\text{Caesar})$

5. $\forall x : \text{Roman}(x) \rightarrow \text{loyalto}(x, \text{Caesar}) \vee \text{hate}(x, \text{Caesar})$

6. $\forall x : \exists y : \text{loyalto}(x, y)$

7. $\forall x : \forall y : \text{person}(x) \wedge \text{ruler}(y) \wedge \text{try-assassinate}(x, y) \rightarrow \neg \text{loyalto}(x, y)$

8. $\text{try-assassinate}(\text{Marcus}, \text{Caesar})$

9. $\forall x : \text{man}(x) \rightarrow \text{person}(x)$



Resolution in FOL

– Convert the given facts into CNF:

1. Man(Marcus)

2. Pompeian(Marcus)

3. $\forall x : \text{Pompeian}(x) \rightarrow \text{Roman}(x) \equiv$

$\forall x : \neg \text{Pompeian}(x) \vee \text{Roman}(x) \equiv$

$\neg \text{Pompeian}(x1) \vee \text{Roman}(x1)$

4. ruler(Caesar)

5. $\forall x : \text{Roman}(x) \rightarrow \text{loyalto}(x, \text{Caesar}) \vee \text{hate}(x, \text{Caesar}) \equiv$

$\forall x : \neg \text{Roman}(x) \vee [\text{loyalto}(x, \text{Caesar}) \vee \text{hate}(x, \text{Caesar})] \equiv$

$\neg \text{Roman}(x2) \vee \text{loyalto}(x2, \text{Caesar}) \vee \text{hate}(x2, \text{Caesar})$



Resolution in FOL

$$6. \forall x : \exists y : \text{loyalto}(x, y) \equiv$$

$$\forall x : \text{loyalto}(x, f(x)) \equiv$$

$$\forall x3 : \text{loyalto}(x3, f(x3)) \equiv$$

$$\text{loyalto}(x3, f(x3))$$

$$7. \forall x : \forall y: \text{person}(x) \wedge \text{ruler}(y) \wedge \text{try-assassinate}(x, y) \rightarrow \neg \text{loyalto}(x, y) \equiv$$

$$\forall x : \forall y: \neg[\text{person}(x) \wedge \text{ruler}(y) \wedge \text{try-assassinate}(x, y)] \vee \neg \text{loyalto}(x, y) \equiv$$

$$\forall x4 : \forall y1: \neg \text{person}(x4) \vee \neg \text{ruler}(y1) \vee \neg \text{try-assassinate}(x4, y1) \vee \neg \text{loyalto}(x4, y1) \equiv$$

$$\neg \text{person}(x4) \vee \neg \text{ruler}(y1) \vee \neg \text{try-assassinate}(x4, y1) \vee \neg \text{loyalto}(x4, y1)$$



Resolution in FOL

8. try-assassinate(Marcus, Caesar)

9. $\forall x : \text{man}(x) \rightarrow \text{person}(x) \equiv$

$\forall x : \neg \text{man}(x) \vee \text{person}(x) \equiv$

$\neg \text{man}(x5) \vee \text{person}(x5)$



Resolution in FOL

– Example: given facts in CNF:

1. $\text{Man}(\text{Marcus})$
2. $\text{Pompeian}(\text{Marcus})$
3. $\neg \text{Pompeian}(x_1) \vee \text{Roman}(x_1)$
4. $\text{ruler}(\text{Caesar})$
5. $\neg \text{Roman}(x_2) \vee \text{loyalto}(x_2, \text{Caesar}) \vee \text{hate}(x_2, \text{Caesar})$
6. $\text{loyalto}(x_3, f(x_3))$
7. $\neg \text{person}(x_4) \vee \neg \text{ruler}(y_1) \vee \neg \text{try-assassinate}(x_4, y_1) \vee \neg \text{loyalto}(x_4, y_1)$
8. $\text{try-assassinate}(\text{Marcus}, \text{Caesar})$
9. $\neg \text{man}(x_5) \vee \text{person}(x_5)$



Resolution in FOL

- Prove that Marcus hate Caesar. That is
 $\text{hate}(\text{Marcus}, \text{Caesar})$

Negate: $\neg \text{hate}(\text{Marcus}, \text{Caesar})$

which is already in CNF



Resolution in FOL

- **Resolution**

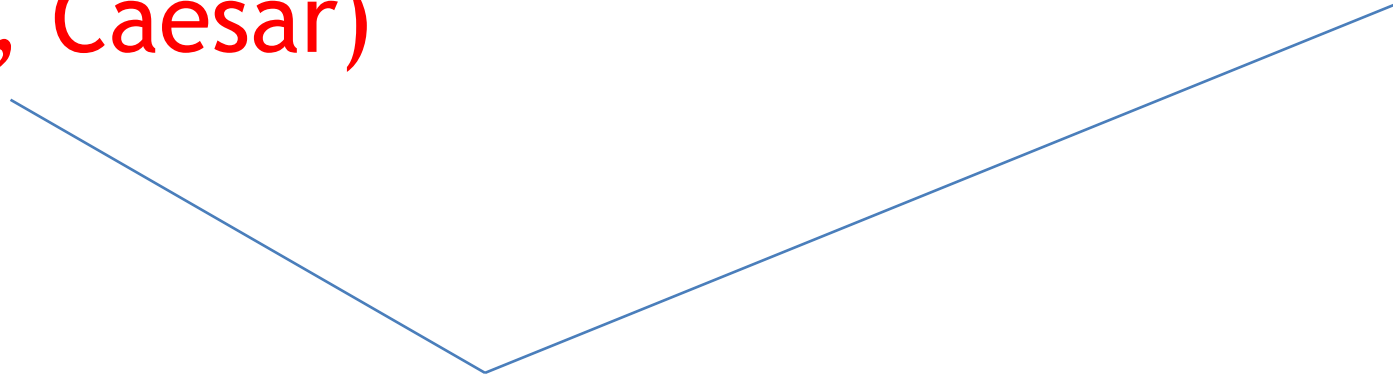
→ hate(Marcus, Caesar)



Resolution in FOL

– hate(Marcus, Caesar)

– Roman(x2) $\vee$ loyalto(x2, Caesar) $\vee$ hate(x2, Caesar)



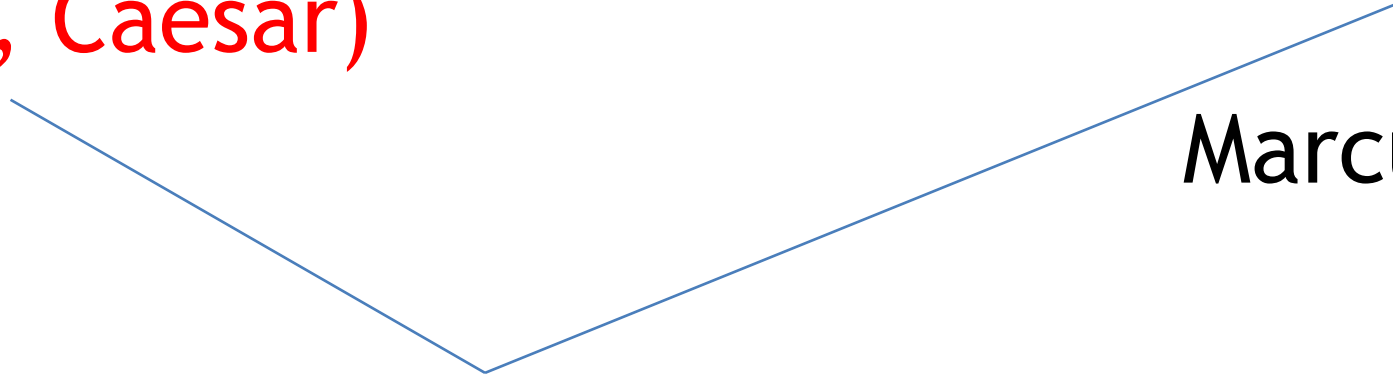
Resolution in FOL

- **Resolution**

– Roman(x2) $\vee$ loyalto(x2, Caesar) $\vee$ hate(x2, Caesar)

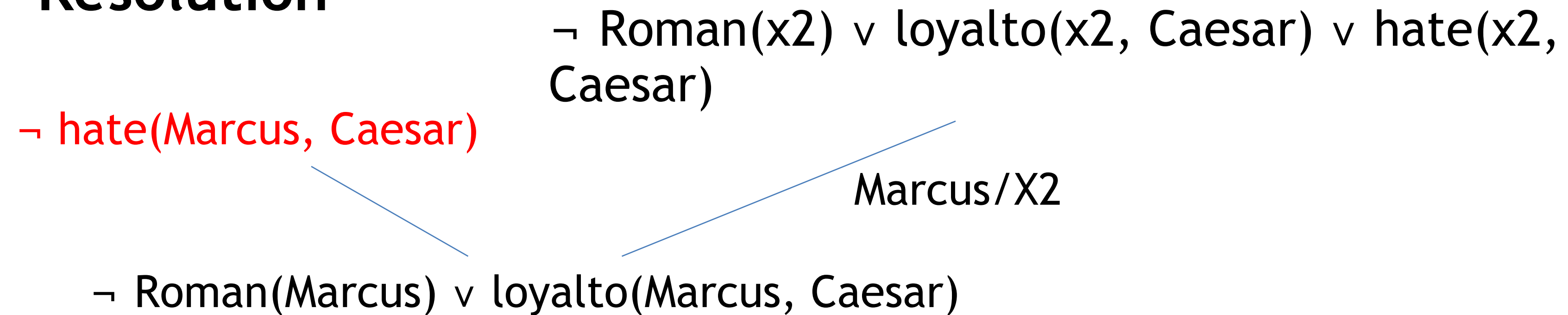
– hate(Marcus, Caesar)

Marcus/X2



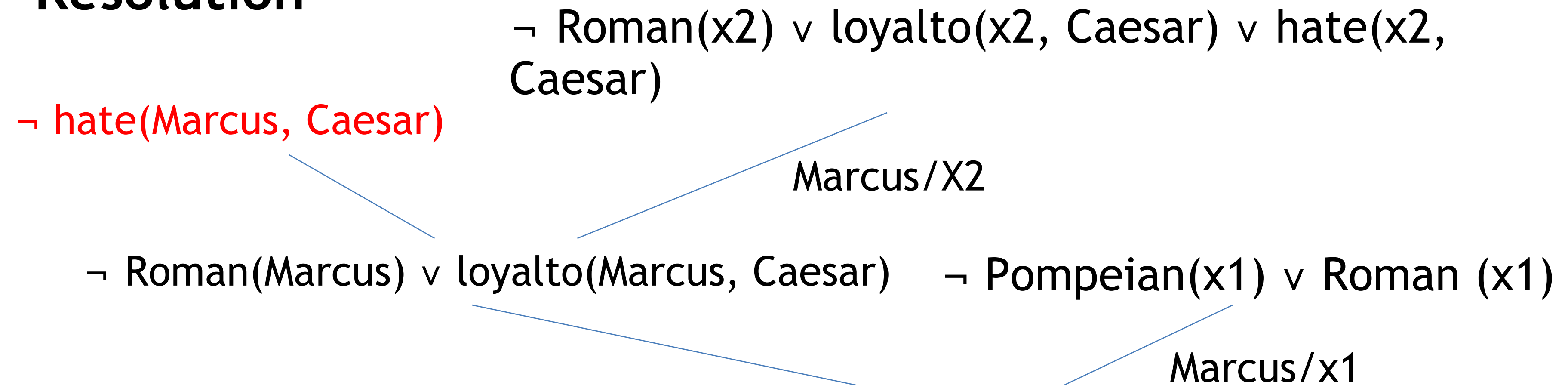
Resolution in FOL

- **Resolution**



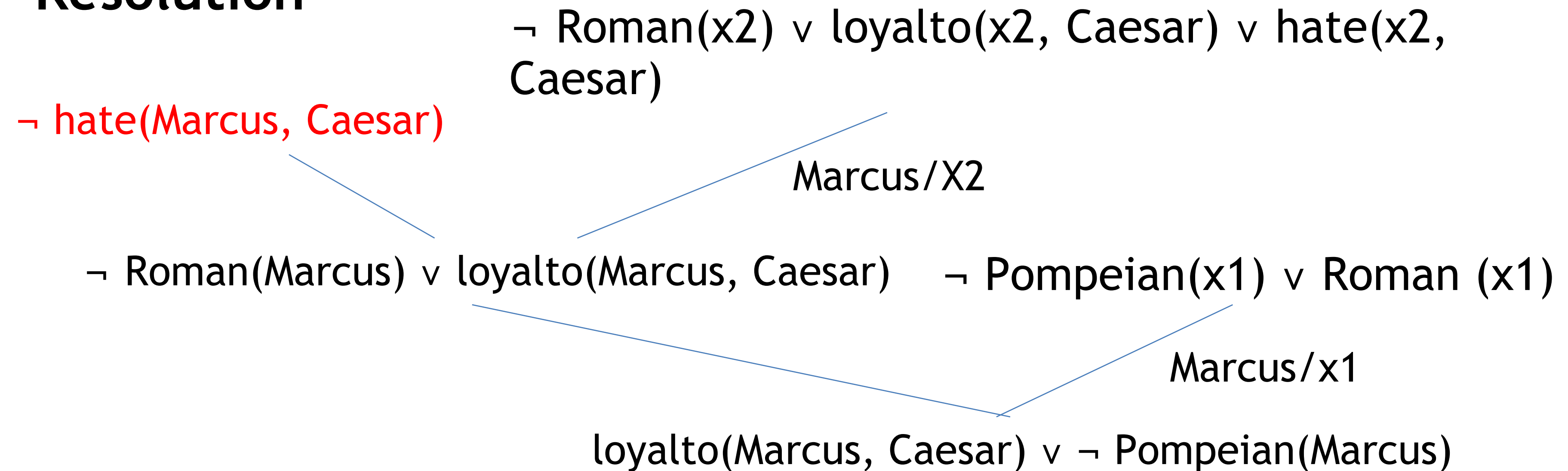
Resolution in FOL

- **Resolution**



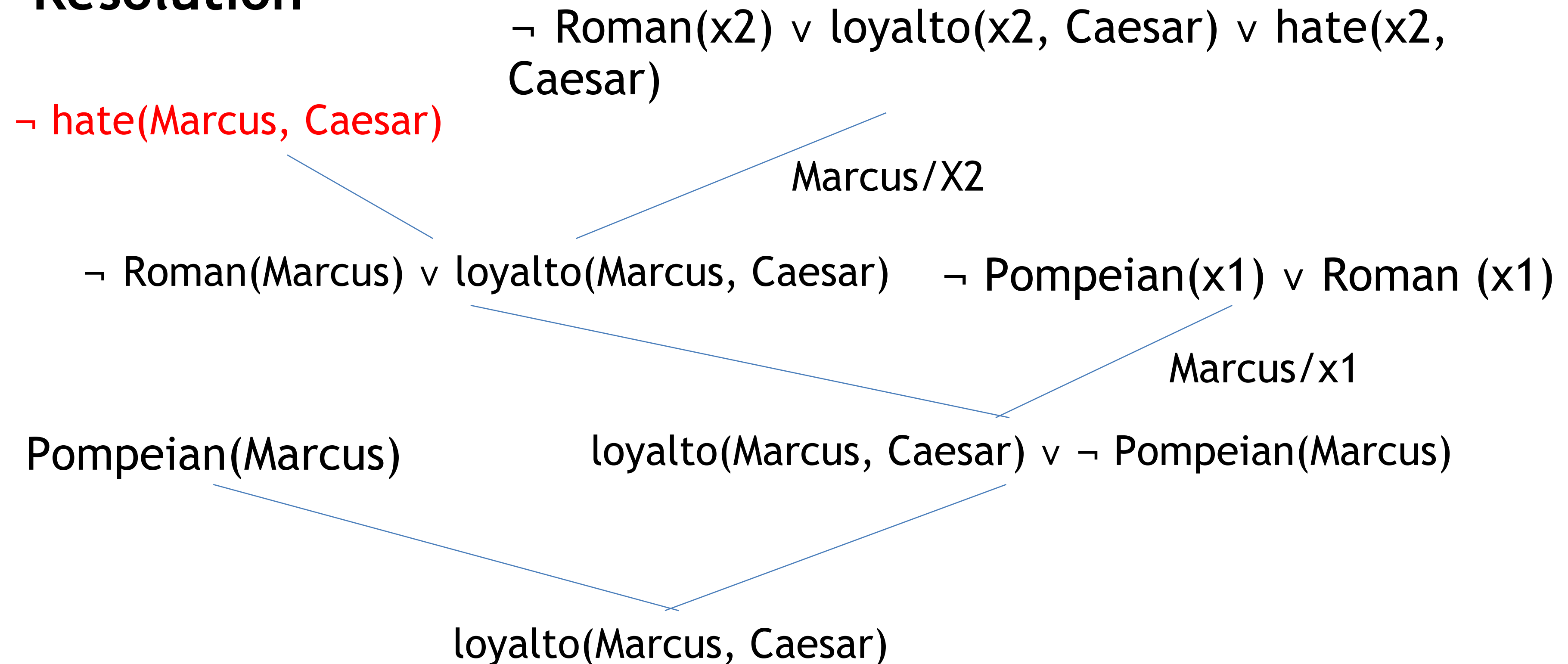
Resolution in FOL

- **Resolution**



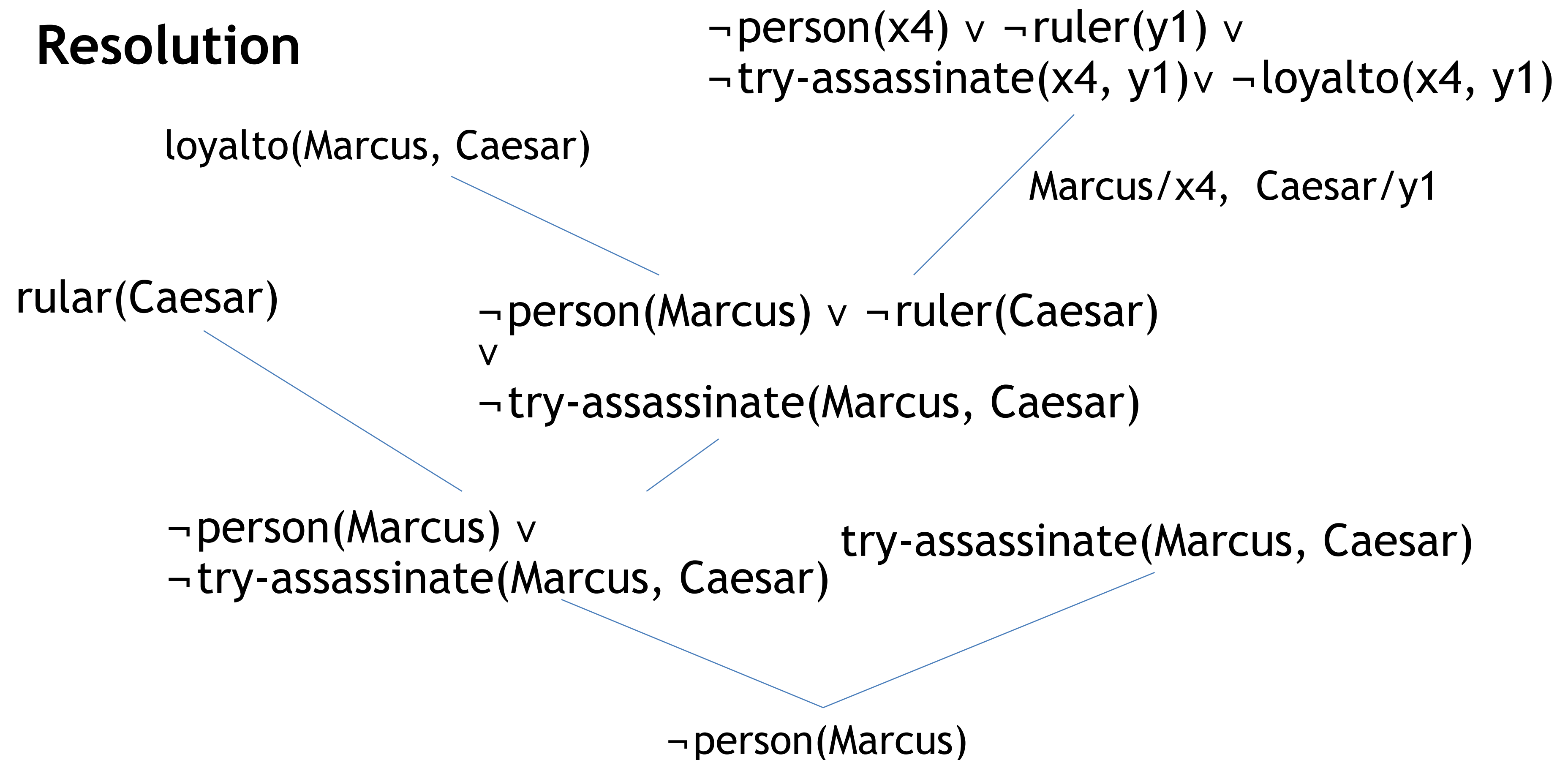
Resolution in FOL

- **Resolution**



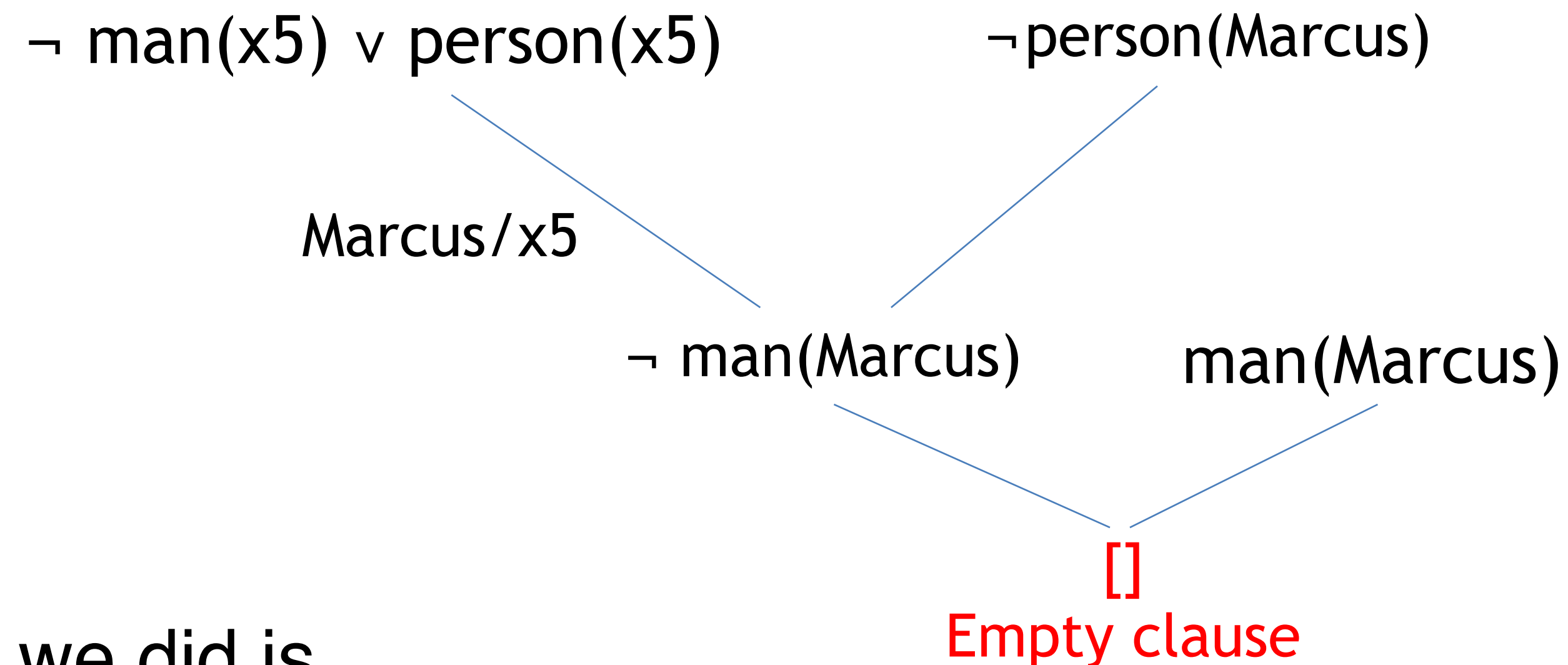
Resolution in FOL

- Resolution**



Resolution in FOL

- Resolution



- What we did is
 - Backward reason
 - Got contradiction, ie $\neg \text{hate}(\text{Marcus}, \text{Caesar})$ was found wrong
 - Therefore, “Marcus hates Caesar” is true.



Resolution in FOL

- **Resolution**
 - Question Answering
 - Resolution can also be used to answer the questions
 - Example: Assume the following facts:
 - John only likes easy courses.
 - All science courses are hard.
 - All the courses in Arts are easy.
 - A101 is an Art course.
 - S201 is a science course.
 - Now, using resolution answer the following:
 - Which course would John like?



Resolution in FOL

- **Resolution**

- Given facts in predicate logic

- John only likes easy courses.

$\forall x: \text{easyCourse}(x) \rightarrow \text{likes}(\text{John}, x)$

- All science courses are hard.

$\forall x: \text{scienceCourse}(x) \rightarrow \text{hardCourse}(x)$

- All the courses in Arts are easy.

$\forall x: \text{artCourse}(x) \rightarrow \text{easyCourse}(x)$

- A101 is an Art course.

$\text{artCourse}(\text{A101})$

- S201 is a science course.

$\text{scienceCourse}(\text{S201})$



Resolution in FOL

- Resolution
 - Given facts in CNF
 - $\neg \text{easyCourse}(x1) \vee \text{likes}(\text{John}, x1)$
 - $\neg \text{scienceCourse}(x2) \vee \text{hardCourse}(x2)$
 - $\neg \text{artCourse}(x3) \vee \text{easyCourse}(x3)$
 - $\text{artCourse}(\text{A101})$
 - $\text{scienceCourse}(\text{S201})$



Resolution in FOL

- Resolution
 - Question clause is

Likes(John, x)

- For this, first prove that John like some course first.

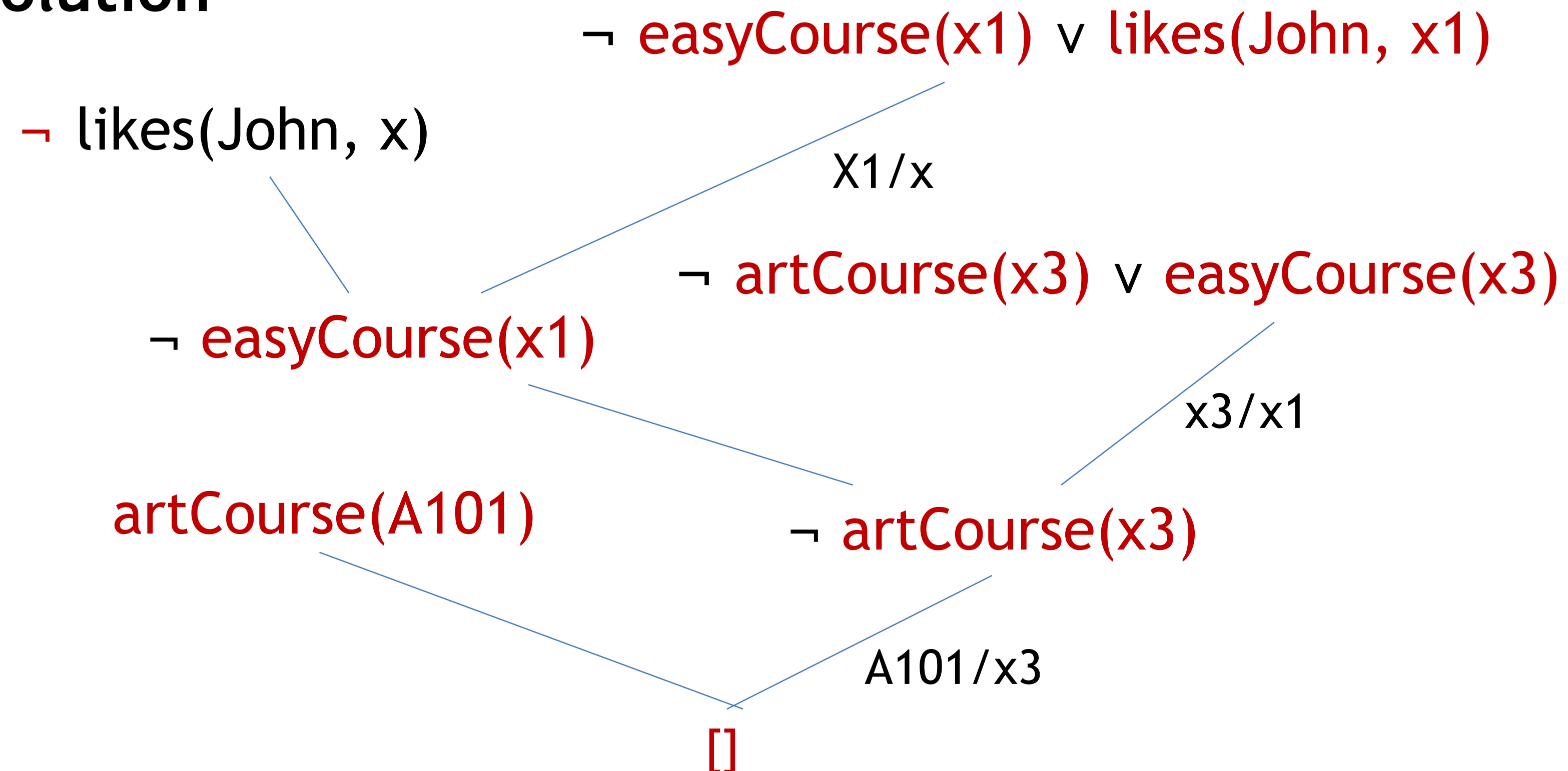
That is,

likes(John, x)



Resolution in FOL

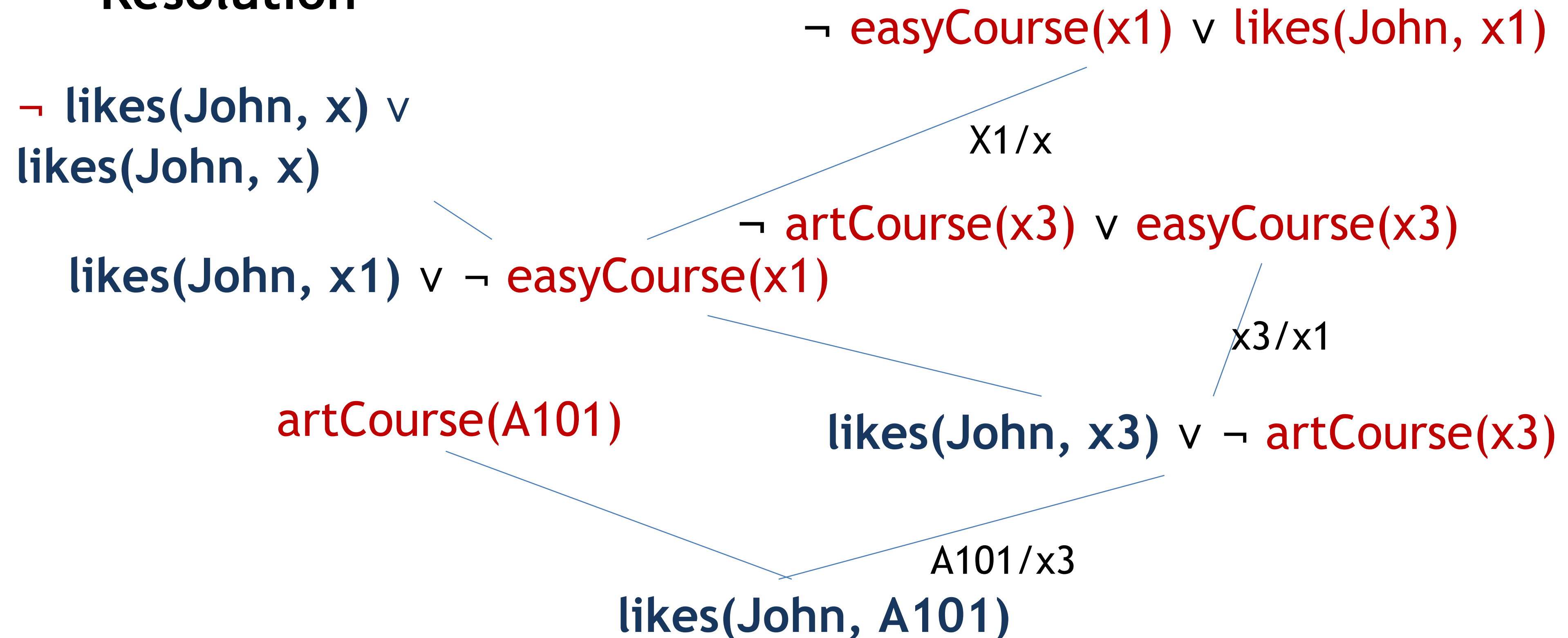
- Resolution



It means **John likes some course x.**
Which course would John like?

Resolution in FOL

- Resolution

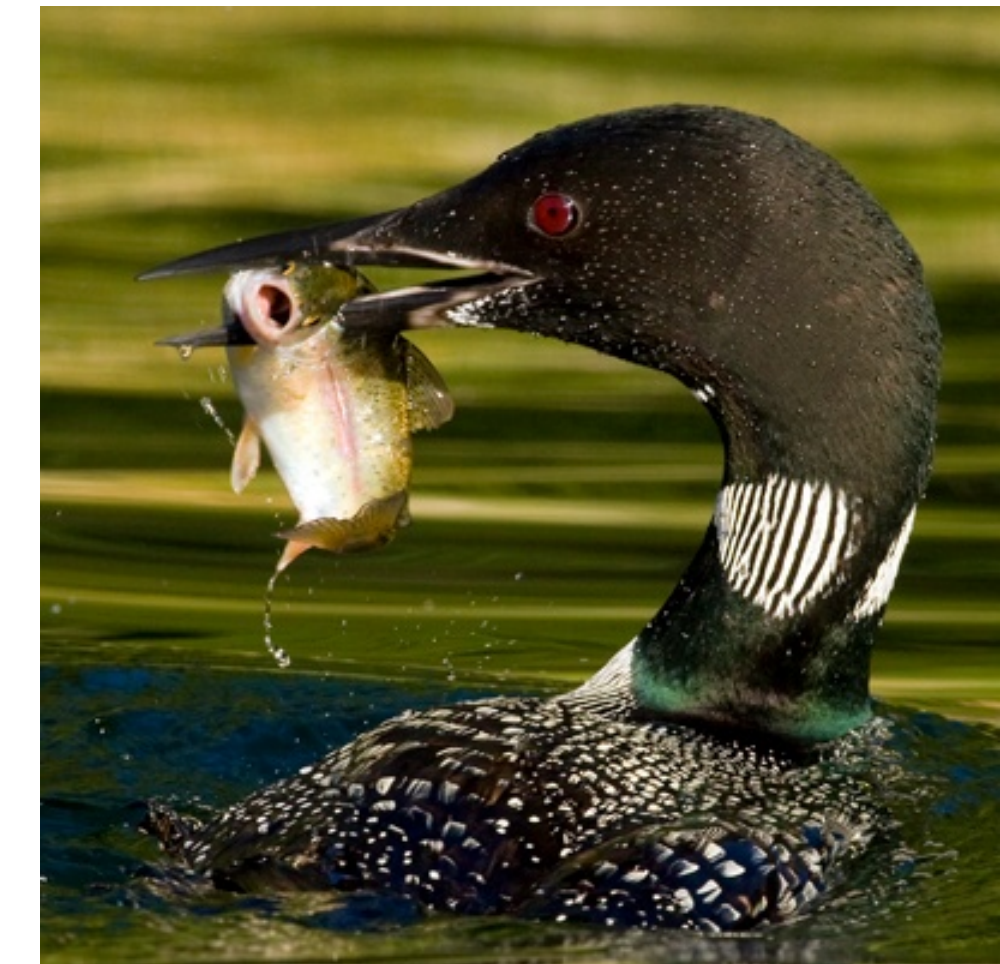


Therefore, John likes Art course A101



Resolution in FOL

- Consider the following axioms.
 - Every bird sleeps in some tree.
 - Every loon is a bird, and every loon is aquatic.
 - Every tree in which any aquatic bird sleeps is beside some lake.
 - Anything that sleeps in anything that is beside any lake eats fish.



Using resolution prove that “Every loon eats fish”.

[Solution: http://www.sc.ehu.es/jiwlucap/Tema3-RA-2013-2014.pdf](http://www.sc.ehu.es/jiwlucap/Tema3-RA-2013-2014.pdf)

Resolution in FOL

- Consider the following axioms.
 - John likes all kind of food.
 - Apples and chicken are food
 - Anything anyone eats and is not killed is food
 - Mary eats peanuts and is still alive
 - Bob eats everything that Mary eats
 - Everyone who is alive is not killed.
 - Everyone who is not killed is alive.

Prove by resolution that John likes peanuts.



Resolution in FOL

- **Resolution**

- facts in predicate logic.

- $\forall x : \text{food}(x) \rightarrow \text{likes}(\text{John}, x)$

- $\text{food}(\text{Apple}) \wedge \text{food}(\text{chicken})$

- $\forall a : \forall b : \text{eats}(a, b) \wedge \sim \text{killed}(a) \rightarrow \text{food}(b)$

- $\text{eats}(\text{Mary}, \text{Peanuts}) \wedge \text{alive}(\text{Mary})$

- $\forall c : \text{eats}(\text{Mary}, c) \rightarrow \text{eats}(\text{Bob}, c)$

- $\forall d : \text{alive}(d) \rightarrow \sim \text{killed}(d)$

- $\forall e : \sim \text{killed}(e) \rightarrow \text{alive}(e)$

- Conclusion: likes (John, Peanuts)**



Resolution in FOL

- Resolution
 - Facts in CNF.
 1. $\sim \text{food}(x) \vee \text{likes}(\text{John}, x)$
 2. $\text{Food}(\text{apple})$
 3. $\text{Food}(\text{chicken})$
 4. $\sim \text{eats}(a, b) \vee \text{killed}(a) \vee \text{food}(b)$
 5. $\text{Eats}(\text{Mary}, \text{Peanuts})$
 6. $\text{Alive}(\text{Mary})$
 7. $\sim \text{eats}(\text{Mary}, c) \vee \text{eats}(\text{Bob}, c)$
 8. $\sim \text{alive}(d) \vee \sim \text{killed}(d)$
 9. $\text{Killed}(e) \vee \text{alive}(e)$
 - To prove: $\text{likes}(\text{John}, \text{Peanuts})$



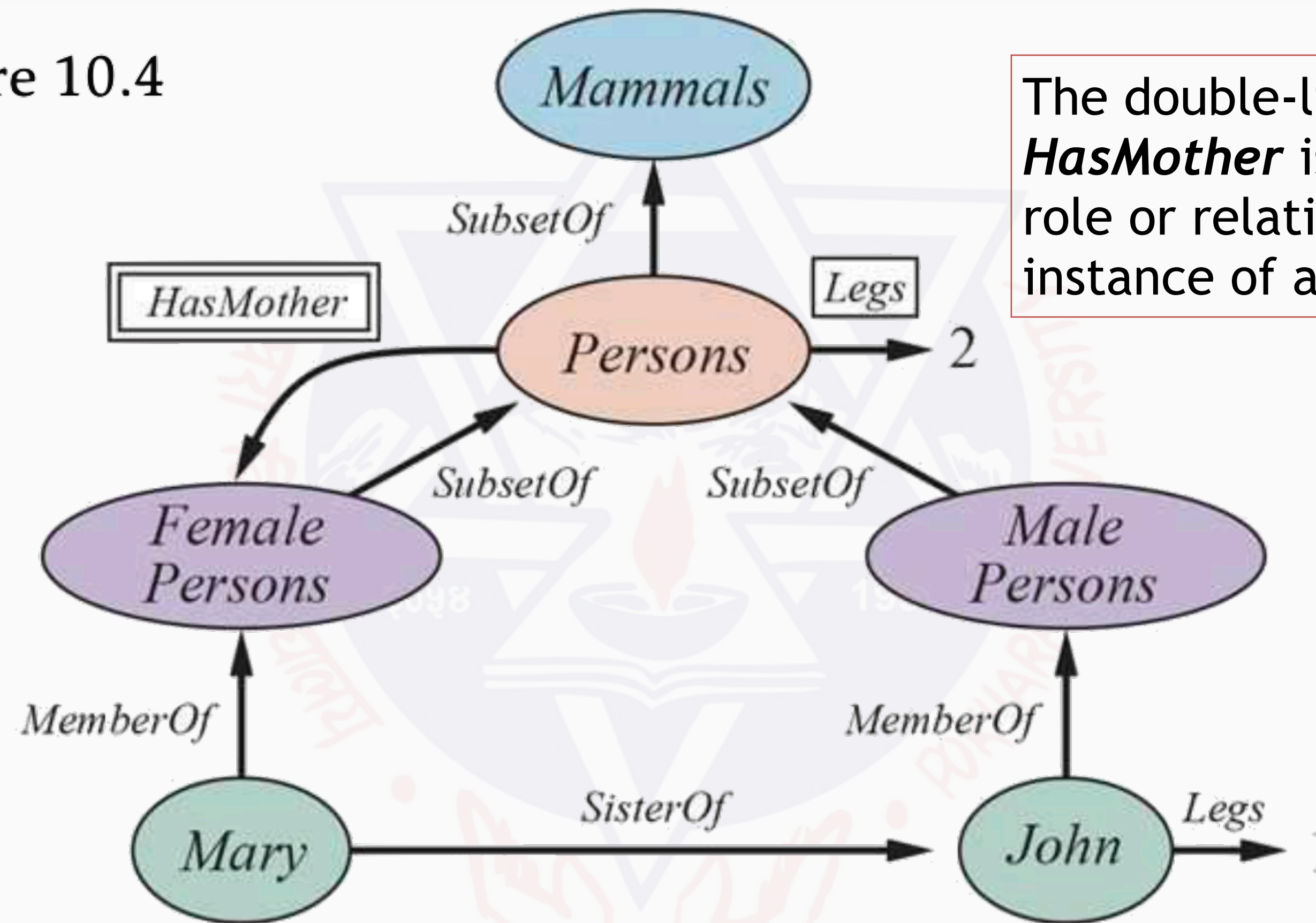
Semantic Networks

- In 1909, Charles S. Peirce proposed a graphical notation of nodes and edges called existential graphs
 - he called it “the logic of the future.”
 - Later it is called as Semantic networks
- Semantic networks are a form of logic.
- There are many variants of semantic networks: all are capable of representing
 - individual objects,
 - categories of objects, and
 - relations among objects
- A typical graphical notation
 - displays object or category names in ovals or boxes, and
 - connects them with labeled links



Semantic Networks

Figure 10.4



The double-lined rectangle indicates that *HasMother* is being treated as a generic role or relation type- (not a specific instance of a relationship)

A semantic network with four objects (John, Mary, 1, and 2) and four categories. Relations are denoted by labeled links.



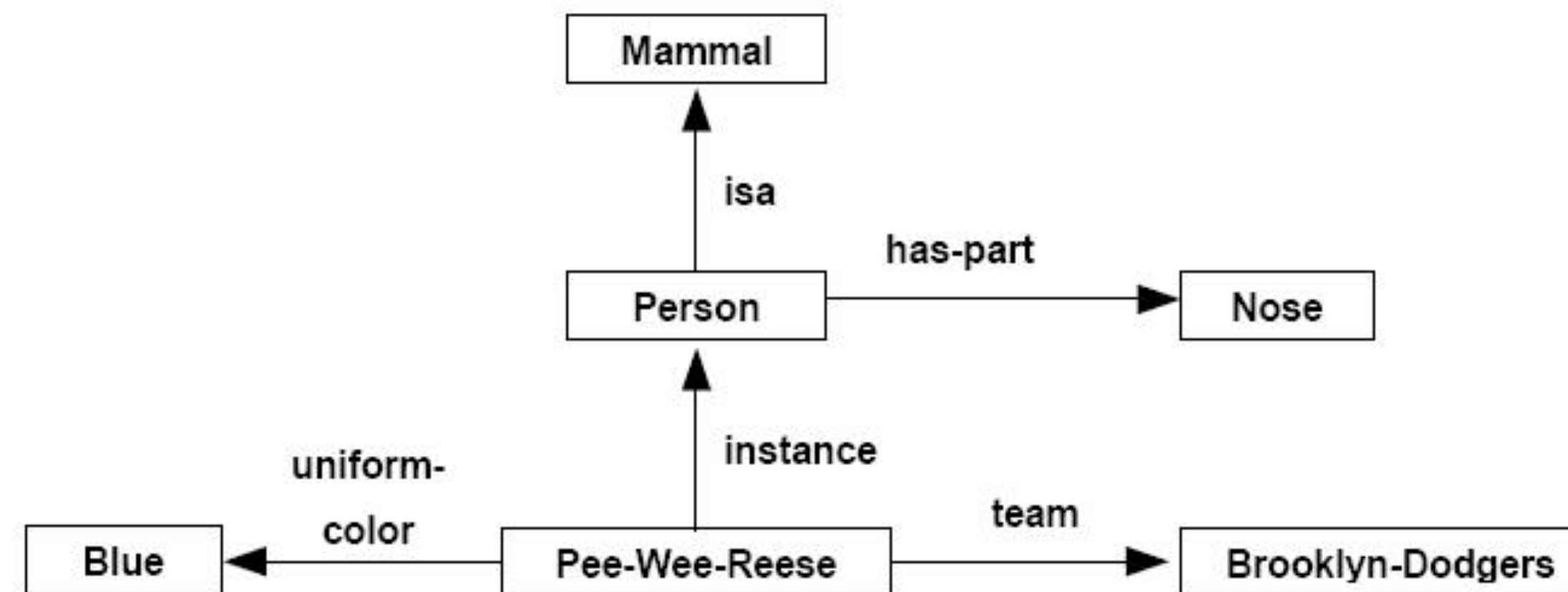
Semantic Nets

- is simple KR scheme
- uses a graph of labeled nodes and labeled directed arcs to encode knowledge
 - Nodes
 - objects, concepts, events
 - represents the information
 - Arcs – relationships between nodes
 - particularly *isa* arcs -
 - allow *inheritance* of properties.



Semantic Nets

- Example: *semantic net*



In logic representation

```
isa(Person, Mammal)
has-part(Person, Nose)
instance(Pee-Wee-Reese, Person)
team(Pee-Wee-Reese, Brooklyn-Dodgers)
uniform-color(Pee-Wee-Reese, Blue)
```



Semantic Nets

- Example: *semantic net*

Tom is a cat.

Tom caught a bird.

Tom is owned by John.

Tom is ginger in colour.

Cats like cream.

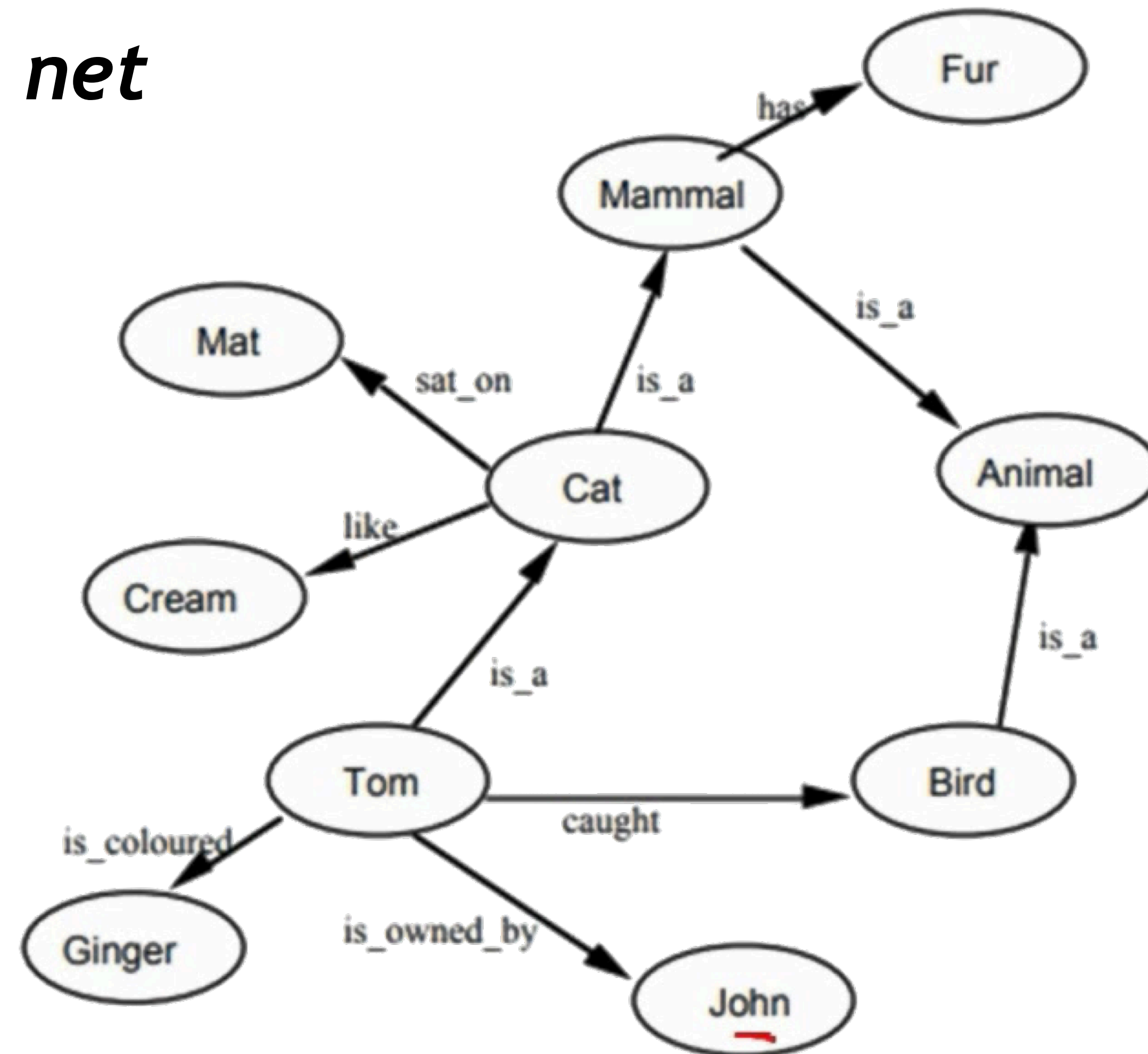
The cat sat on the mat.

A cat is a mammal.

A bird is an animal.

All mammals are animals.

Mammals have fur.



Semantic Nets

- Benefits of *semantic net*
 - Easy to visualize
 - Formal definitions of semantic networks have been developed.
 - Related knowledge is easily clustered.
 - Efficient in space requirements
 - Objects represented only once
 - Relationships handled by pointers





THANK YOU